Core Maintenance on Dynamic Multilayer Graphs

Yuhan Zhou[1], Chuanyu Liu[1], Qing Liu[1,2], Chengyang Luo[1]
[1]Zhejiang University, [2]Zhejiang Key Laboratory of Big Data Intelligent Computing
{zhou_yh,liucy219,qingliucs,luocy1017}@zju.edu.cn

ABSTRACT

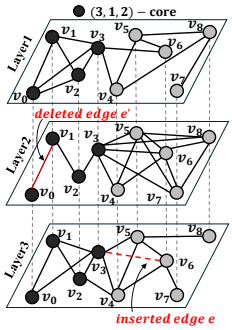
Multilayer core is a fundamental cohesive subgraph model for multilayer graph analysis. Existing multilayer core decomposition methods, which aim to compute all multilayer cores of a given multilayer graph, mainly focus on static graphs. However, real-world multilayer graphs evolve continuously. Recomputing all multilayer cores from scratch after each graph update is computationally expensive. Therefore, in this paper, we systematically explore the multilayer core maintenance problem on dynamic multilayer graphs. To address this problem, we propose efficient maintenance algorithms that first identify the affected scope for graph updates, and then compute the updated multilayer core through local convergence. To enhance efficiency, we reduce the affected scope through theoretical analysis and accelerate the local convergence procedure with a series of optimizations. Moreover, we extend our algorithms to batch updates with a hybrid edge grouping strategy, which enables all edges within the same group to be processed simultaneously. Extensive experiments on real-world multilayer graphs demonstrate that our proposed algorithms are up to 3 orders of magnitude faster than the state-of-the-art method.

PVLDB Reference Format:
Yuhan Zhou, Chuanyu Liu, Qing Liu, Chengyang Luo. Core Maintenance on Dynamic Multilayer Graphs. PVLDB, 14(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX
PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/ZJU-DAILY/MCM.

1 INTRODUCTION

Real-world entities often exhibit multiple types of relationships [31, 48], which can be naturally modeled as multilayer graphs. In such graph structures, vertices are shared across different layers, while the edges in each layer represent a specific type of relationship [39, 51]. Multilayer graphs have been widely adopted in various application scenarios, such as social networks, collaboration networks, and financial networks [19, 45]. Figure 1(a) shows an example of a multilayer graph.

As a fundamental task in graph analysis, cohesive subgraph mining has been extensively studied [11, 25, 28]. One of the most representative models is k-core, which requires each vertex in the subgraph to have at least k neighbors [7, 44]. To extend the k-core model to multilayer graphs, Azimi et al. [2] proposed the multilayer core, also called $\mathbf{k}$-core, where $\mathbf{k} = [k_\ell]_{\ell \in L}$ requires that the



(a) A multilayer graph $\mathcal{G}$

v	MCV Set
v_0	(3,1,2)
v_1	(3,1,2)
v_2	(3,1,2)
v_3	(1,4,1)(3,1,2)(2,3,2)
v_4	(2,3,2)
v_5	(1,4,1)(2,3,2)
v_6	(1,4,1) (2,3,2)
v_7	(1,4,1) (2,3,1)
v_8	(1,4,1)

(b) MCVs of $\mathcal{G}$

v	MCV Set
v_0	(3,1,2)
v_1	(3,1,2)
v_2	(3,1,2)
v_3	(1,4,1)(3,1,2)(2,3,3)
v_4	(2,3,3)
v_5	(1,4,1) (2,3,3)
v_6	(1,4,1) (2,3,3)
v_7	(1,4,1) (2,3,1)
v_8	(1,4,1)

(c) MCVs of $\mathcal{G} \cup \{e\}$

Figure 1: An example of multilayer core maintenance

subgraph induced on each layer $\ell \in L$ forms a k_ℓ-core (L is the layer set). In Figure 1(a), the subgraph induced by $\{v_0, v_1, v_2, v_3\}$ is a $\mathbf{k}$-core with $\mathbf{k} = (3, 1, 2)$. The $\mathbf{k}$-core has a wide range of applications [15, 34], including social recommendation, collaboration analysis, and fraud detection.

To efficiently compute $\mathbf{k}$-cores, previous studies [15, 34] investigated the multilayer core decomposition problem, which aims to enumerate all $\mathbf{k}$-cores on a given multilayer graph $\mathcal{G}$ and return the maximal coreness vectors (MCVs) of each vertex [15]. A vector $\mathbf{k}$ is a coreness vector of a vertex v if v belongs to a $\mathbf{k}$-core. For two vectors $\mathbf{k}$ and $\mathbf{k}'$, $\mathbf{k}$ is dominated by $\mathbf{k}'$ if $k_\ell \leq k'_\ell$ for all $\ell \in L$. A coreness vector is maximal if it is not dominated by any other coreness vector of the same vertex. Figure 1(b) illustrates the results of the multilayer core decomposition of $\mathcal{G}$. The MCV of v_4 is $\mathbf{k}^* = (2, 3, 2)$, indicating that it can be contained in any $\mathbf{k}$-core with $\mathbf{k}$ dominated by $\mathbf{k}^*$. Based on the results of the multilayer core decomposition, we can efficiently retrieve the $\mathbf{k}$-core by collecting the vertices with an MCV $\mathbf{k}^*$ dominating $\mathbf{k}$.

Despite the advances in existing studies, they are primarily designed for static multilayer graphs, whereas real-world networks are highly dynamic [36, 46]. For example, user connections in social networks constantly change; researchers in collaboration networks frequently form new partnerships; and malicious actors in fraud networks continually adapt their interactions to evade detection. These dynamic changes highlight the need for efficient algorithms to update $\mathbf{k}$-cores after graph changes. Therefore, for the first time, we study multilayer core maintenance on dynamic multilayer graphs and theoretically analyze its update boundedness. Specifically, given a multilayer graph and an updated edge, the goal is to maintain the MCVs of vertices. Multilayer core maintenance benefits various dynamic graph analysis tasks by enabling efficient updates of cohesive structures in evolving multilayer networks, with applications including dynamic product-group discovery in co-purchasing networks [29], functional module analysis in biological networks [42], and evolving community tracking in social networks [14].

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

A straightforward solution for multilayer core maintenance is to invoke existing multilayer core decomposition algorithms after each graph update. However, it is inefficient because it recomputes all $\mathbf{k}$-cores from scratch, even though only a small fraction of vertices and edges are typically affected in real-world scenarios [53]. In the literature, various core maintenance algorithms [4, 6, 12, 23, 43] have been proposed for different types of graphs. A natural idea is to apply existing core maintenance techniques independently to each layer. However, it cannot correctly solve the multilayer core maintenance problem due to a fundamental model mismatch. In multilayer graphs, unchanged single-layer coreness does not imply unchanged MCVs. This is because single-layer coreness characterizes each layer independently, whereas an MCV requires the degree constraints on all layers to be simultaneously satisfied by the same vertex set. For example, in Figure 1, after removing edge $e' = (v_0, v_1, 2)$ from layer 2, the single-layer coreness of v_1 remains 3, 1, and 2 on layers 1-3, respectively. However, the MCV of v_1 changes because v_1 can no longer belong to a $(3, 1, 2)$-core. Therefore, the vertices with changed MCVs cannot be inferred from the vertices with changed single-layer coreness. Even if all per-layer coreness values are correctly maintained, their naive combination cannot correctly determine the MCVs. These limitations necessitate dedicated maintenance techniques tailored to multilayer cores.

To address these limitations, we propose efficient multilayer core maintenance algorithms. Following the paradigm of existing core maintenance methods [8, 16, 22], our algorithms identify the affected scope and locally update the MCVs, while building on novel theoretical analysis and optimizations tailored to multilayer graphs. For the affected scope identification procedure, we analyze the impacts of edge insertion and deletion. For edge insertion, we prove that each updated MCV is dominated by an original MCV whose value is increased by one on the updated layer. Based on this bound, we initialize affected vectors from the inserted edge and iteratively propagate the incremental impact to all potentially influenced vertices. For edge deletion, we observe that the vector obtained by decreasing the deleted-layer value of the original MCV by one remains a valid coreness vector in the updated graph. Based on this, we track original MCVs that may lose dominating MCVs from their neighbors, and iteratively propagate such losses using the valid coreness vector as a pruning condition to identify affected vertices. Consequently, the procedure returns the affected vertices and upper-bound vectors dominating all updated MCVs.

After obtaining the affected vertices and their upper-bound vectors, we compute the updated MCVs through a local convergence procedure. Starting from the upper-bound vectors, the procedure repeatedly validates candidate vectors and refines invalid ones until convergence. Specifically, an invalid vector is split greedily by decreasing the value of each dimension, and the resulting vectors are further validated. The process terminates when all remaining vectors are valid MCVs. To improve efficiency, we optimize both validation and refinement by reusing historical validation results and shrinking the search space through repeated vector filtering, binary state skipping, and redundant vector pruning.

Furthermore, we extend our maintenance algorithm to support batch updates through a hybrid edge grouping strategy. The key observation is that multiple edge updates can be processed simultaneously if they preserve the update bound derived for a single-edge update. To this end, we first introduce disjoint edge groups, in which no two updated edges share an endpoint. Consequently, each vertex gains or loses at most one incident edge on the updated layer, ensuring that the single-edge update bound remains valid. To prevent performance degradation when many updated edges share common endpoints, we further introduce joint edge groups, which permit shared endpoints whose MCV values on the updated layer are strictly larger than those of their opposite endpoints. Based on these two grouping rules, we identify the affected scope and perform local convergence group by group to update MCVs efficiently. The main contributions of this paper are as follows:

- To the best of our knowledge, this is the first work to investigate the multilayer core maintenance problem on dynamic multilayer graphs with a theoretical analysis of its boundedness.
- We devise efficient two-phase multilayer core maintenance algorithms and a set of optimization techniques, supported by thorough theoretical analysis.
- We design a hybrid edge grouping strategy for our maintenance algorithms to efficiently handle batch updates.
- We conduct extensive experiments on real-world multilayer graphs, demonstrating the efficiency of our proposed algorithms and the effectiveness of optimizations.

2 RELATED WORK

Core maintenance. The core maintenance aims to update the coreness of vertices when the graph is updated. Traverse-based methods [30, 43] update the affected region through graph traversal. k-order-based methods [17, 56] further accelerate the process by utilizing the peeling order generated during core decomposition. Convergence-based methods [8, 16, 22] iteratively refine coreness values until convergence. Additionally, several techniques [21, 27, 50] have been proposed to partition updated edges for batch updates. Core maintenance has also been studied on other graphs, including weighted graphs [54, 57], directed graphs [12, 32], bipartite graphs [4, 40, 53], uncertain graphs [6], temporal graphs [3, 52], and hypergraphs [10, 23, 38]. Despite the advances of existing works, they are designed for specific graphs and core models, making them unsuitable for multilayer core maintenance.

Cohesive subgraphs in multilayer graphs. Various multilayer cohesive subgraph models have been proposed, including core-based models [2, 15, 20, 33–35, 39, 45, 51, 59], truss-based models [5, 24, 37], and clique-based models [26, 49]. Core-based models are one of the most representative categories. Specifically, the $\mathbf{k}$-core with $\mathbf{k} = [k_\ell]_{\ell \in L}$ enforces that the subgraph induced in each layer $\ell \in L$ forms a k_ℓ-core [2, 15, 34], which is the focus of this paper. Additionally, several variants have been proposed by relaxing these conditions or incorporating additional constraints, including d-CC [33, 59], FirmCore [20], FocusCore [51], gCore [35], (k, d)-core [45], and SynCore [39]. However, existing works only consider static multilayer graphs, whereas this paper focuses on dynamic multilayer graphs.

3 PROBLEM FORMULATION

A multilayer graph can be represented as $\mathcal{G} = (V, E, L)$, where V is the set of vertices, $E \subseteq V \times V \times L$ is the set of edges, and L is the set of layers. For each $\ell \in L$, the corresponding single-layer graph in layer

ℓ is denoted by $\mathcal{G}_\ell = (V, E_\ell)$. Note that graphs on different layers share the same vertex set, while their edge sets differ from one another. Let $C \subseteq V$ be a subset of vertices. The induced multilayer subgraph of $\mathcal{G}$ on C is defined as $\mathcal{G}[C] = (C, E[C], L)$. Similarly, for each layer ℓ, the induced subgraph of $\mathcal{G}_\ell$ on C is denoted by $\mathcal{G}_\ell[C] = (C, E_\ell[C])$ with $E_\ell[C] = \{(u, v) \in E_\ell \mid u, v \in C\}$. For a vertex $v \in V$, we use $N_{\mathcal{G}}(v, \ell)$ and $deg_{\mathcal{G}}(v, \ell)$ to denote v's neighbors and degree in $\mathcal{G}_\ell$, respectively. Correspondingly, $N_{\mathcal{G}}(v) = \bigcup_{\ell \in L} N_{\mathcal{G}}(v, \ell)$. We use $e = (u, w, \ell)$ to denote a single updated edge on layer ℓ. The updated multilayer graph after inserting or deleting e is denoted by $\mathcal{G} \cup \{e\}$ and $\mathcal{G} \setminus \{e\}$, respectively. Next, we introduce the multilayer core and subsequently formulate the problem.

Definition 1. *(Multilayer $\mathbf{k}$-core [2, 15, 34]). Given a multilayer graph $\mathcal{G} = (V, E, L)$ and an $|L|$-dimensional integer vector $\mathbf{k} = [k_\ell]_{\ell \in L}$, the multilayer $\mathbf{k}$-core of $\mathcal{G}$ is a subgraph $\mathcal{G}[C] \subseteq \mathcal{G}$ such that $\forall v \in C \subseteq V, \ell \in L, deg_{\mathcal{G}[C]}(v, \ell) \geq k_\ell$.*

Definition 2. *(Dominance Relations). Given two $|L|$-dimensional integer vectors $\mathbf{k} = [k_\ell]_{\ell \in L}$ and $\mathbf{k}' = [k'_\ell]_{\ell \in L}$,*

(1) $\mathbf{k}$ dominates $\mathbf{k}'$ (denoted by $\mathbf{k} \succeq \mathbf{k}'$): $\forall \ell \in L, k_\ell \geq k'_\ell$;

(2) $\mathbf{k}$ strongly dominates $\mathbf{k}'$ (denoted by $\mathbf{k} \succ \mathbf{k}'$): $\mathbf{k} \succeq \mathbf{k}'$ and $\exists \ell \in L, k_\ell > k'_\ell$;

(3) if $\mathbf{k}$ does not dominate $\mathbf{k}'$, we denote it by $\mathbf{k} \not\succeq \mathbf{k}'$;

(4) $\mathbf{k}$ and $\mathbf{k}'$ are incomparable: $\mathbf{k} \not\succeq \mathbf{k}'$ and $\mathbf{k}' \not\succeq \mathbf{k}$.

Definition 3. *(Maximal Coreness Vector (MCV) [15]). Given a multilayer graph $\mathcal{G} = (V, E, L)$, an $|L|$-dimensional integer vector $\mathbf{k}^* = [k^*_\ell]_{\ell \in L}$, and a vertex $v \in V$, $\mathbf{k}^*$ is a coreness vector of v if there exists a non-empty $\mathbf{k}^*$-core in $\mathcal{G}$ containing v and $\mathbf{k}^*$ is a maximal coreness vector of v if there does not exist any coreness vector $\mathbf{k}'$ of v satisfying $\mathbf{k}' \succ \mathbf{k}^*$.*

A vertex v may possess multiple MCVs that are pairwise incomparable. We denote the set of all MCVs of v on graph $\mathcal{G}$ as $\Phi_{\mathcal{G}}(v)$. Given a multilayer graph $\mathcal{G}$, we can compute $\Phi_{\mathcal{G}}(v)$ for each vertex v using the multilayer core decomposition algorithms [15, 34]. When edges are inserted into or removed from $\mathcal{G}$, $\Phi_{\mathcal{G}}(v)$ may change, thereby calling for efficient maintenance strategies. Based on this, we formally define our problem below.

Problem 1. *(Multilayer Core Maintenance). Given a multilayer graph $\mathcal{G} = (V, E, L)$, an edge $e = (u, w, \ell)$ to be inserted/deleted, and $\Phi_{\mathcal{G}}(v)$ for each vertex v of $\mathcal{G}$, the problem of multilayer core maintenance is to compute the updated MCV set $\Phi_{\mathcal{G}'}(v)$ for each vertex v based on $\Phi_{\mathcal{G}}(v)$. Here, $\mathcal{G}' = (V, E', L)$ denotes the updated multilayer graph, where $E' = E \cup \{e\}$ for edge insertion and $E' = E \setminus \{e\}$ for edge deletion.*

For example, Figure 1(b) illustrates the multilayer core decomposition result of $\mathcal{G}$ in Figure 1(a). The corresponding maintenance result after inserting edge e is shown in Figure 1(c), where the updated MCVs of vertices v_3, v_4, v_5, and v_6 are highlighted in red. It is worth noting that multilayer graphs are closely related to heterogeneous graphs. When multiple relation types involve the same type of vertices, they can be mapped to different layers sharing a common vertex set, allowing our maintenance approach to be directly applied. However, for heterogeneous graphs involving multiple vertex types, dedicated core models [13, 18] are required to process connections between cross-type vertices. To the best of our knowledge, core maintenance on such heterogeneous graphs remains unexplored, and we leave this direction for future work.

Input Space Complexity. The input space of $\mathcal{G}$ is $O(|V| + |E|)$. Since the ℓ-th dimension of an MCV is bounded by $deg_{\mathcal{G}}(v, \ell)$ and each MCV contains $|L|$ dimensions, storing $\Phi_{\mathcal{G}}(v)$ requires $O\big(|L| \prod_{\ell \in L}(deg_{\mathcal{G}}(v, \ell) + 1)\big)$ space in the worst case. Overall, the input space is $O\big(|V| + |E| + |L| \sum_{v \in V} \prod_{\ell \in L}(deg_{\mathcal{G}}(v, \ell) + 1)\big)$. It is noted that MCVs of each vertex are required to be pairwise incomparable. Therefore, the actual space consumption is typically much smaller than the theoretical worst case.

Boundedness analysis. Following existing works [53, 55], we adopt the model of locally persistent (LP) algorithm for boundedness analysis. An LP algorithm associates each vertex with a local storage block containing auxiliary status and pointers to adjacent vertices. Upon an edge update, the algorithm starts from the updated edge endpoints and accesses other vertices only through local pointers. Let CHANGED contain the updated edge endpoints and vertices whose MCV sets change. For a multilayer graph $\mathcal{G} = (V, E, L)$, we define the c-hop neighborhood on the aggregate graph $G^{\cup} = (V, \bigcup_{\ell \in L} E_\ell)$, where the distance between two vertices is their shortest-path distance in $G^{\cup}$. Let $N_c(\text{CHANGED}, G^{\cup})$ denote all vertices within distance c from any vertices in CHANGED. A problem under the LP algorithms is bounded if there exists a polynomial function f and a fixed positive integer c such that its update cost is $O(f(|N_c(\text{CHANGED}, G^{\cup})|))$; otherwise, it is unbounded. Here, c is a constant independent of the graph and update.

Theorem 1. *The multilayer core maintenance problem is unbounded under the model of LP algorithms.*

Proof Sketch. We prove it by contradiction. We construct a multilayer graph consisting of two long vertex-disjoint paths, with two single-edge insertions ΔE_1 and ΔE_2 placed at opposite ends of paths on the same layer. Consider the two updates applied to the same initial graph $\mathcal{G}$ independently. The MCVs remain the same after each independent update, and each update requires only $O(1)$ accesses under the boundedness assumption. However, when ΔE_1 and ΔE_2 are applied sequentially, they form a cycle on the updated layer and change the MCVs of all vertices along the paths. Therefore, the access traces of applying ΔE_1 and ΔE_2 on $\mathcal{G}$ independently must intersect; otherwise, applying the update ΔE_2 on $\mathcal{G}$ and $\mathcal{G} \cup \Delta E_1$ will produce the same results. Let n denote the length of the paths, which can be chosen arbitrarily large. Since the two updated edges are separated by $\Omega(n)$ local steps, such an intersection requires at least one update to perform $\Omega(n)$ accesses, contradicting its constant-cost bound. □

Note. Due to space limitations, some detailed proofs of this paper are provided in the appendix of our technical report [58].

4 OVERVIEW OF ALGORITHM

In this section, we present the overview of our multilayer core maintenance algorithm. The maintenance algorithm consists of two phases: *affected scope identification* and *local convergence*. The first phase identifies candidate vertices whose MCVs may change, and the second phase computes the updated MCVs for these vertices.

Let $\overline{\Phi}_{\mathcal{G}}(v)$ be the set of incomparable upper-bound vectors of $\Phi_{\mathcal{G}}(v)$ such that for all $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists $\mathbf{k}' \in \overline{\Phi}_{\mathcal{G}}(v)$, $\mathbf{k}' \succeq$

Algorithm 1: Multilayer Core Maintenance (MCM)

Input: A multilayer graph $\mathcal{G}$; input MCV sets $\Phi_{\mathcal{G}}(\cdot)$; updated edge $e = (u, w, \ell)$; operation $op \in \{\text{insert}, \text{delete}\}$
Output: Updated MCV sets $\Phi_{\mathcal{G}'}(\cdot)$

1 $\mathcal{G}' \leftarrow \mathcal{G} \cup \{e\}$ if op = insert; otherwise $\mathcal{G}' \leftarrow \mathcal{G} \setminus \{e\}$;
2 $(\mathcal{A}(e), \overline{\Phi}_{\mathcal{G}'}(\cdot)) \leftarrow \text{IdentifyScope}(\mathcal{G}', \Phi_{\mathcal{G}}(\cdot), e, op)$;
3 $\Phi_{\mathcal{G}'}(\cdot) \leftarrow \text{LocalConvergence}(\mathcal{G}', \overline{\Phi}_{\mathcal{G}'}(\cdot), \mathcal{A}(e))$;
4 **return** $\Phi_{\mathcal{G}'}(\cdot)$;

$\mathbf{k}$. We say $\mathbf{k}'$ is an upper-bound vector of $\mathbf{k}$. Algorithm 1 shows the overview of our maintenance algorithm. The operation op specifies whether e is inserted or deleted. Accordingly, the algorithm first constructs the updated graph (line 1) and then invokes `IdentifyScope` to obtain the affected vertex set $\mathcal{A}(e)$ and the upper-bound vector sets $\overline{\Phi}_{\mathcal{G}'}(\cdot)$ (line 2). Finally, the algorithm invokes `LocalConvergence` to refine the upper-bound vector sets into the updated MCV sets $\Phi_{\mathcal{G}'}(\cdot)$ and returns them (lines 3-4).

Next, we introduce the procedure for affected scope identification and local convergence.

5 AFFECTED SCOPE IDENTIFICATION

In this section, we analyze the affected scope for edge insertion and deletion, respectively, and then introduce the procedure to identify the affected vertices and their upper-bound vector sets.

5.1 Edge Insertion

In this subsection, we discuss the affected bound and affected scope under single-edge insertion.

Affected bound. For single-edge insertion, we observe that every updated MCV can be bounded by increasing only the inserted-layer dimension of some coreness vector in the original graph by one.

Lemma 1 (Edge Insertion Bound). *Let $\mathcal{G}' = \mathcal{G} \cup \{e\}$, where e is inserted on layer ℓ. For any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$, there exists a coreness vector $\mathbf{k}$ of v in $\mathcal{G}$ such that $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$, where $\mathbf{e}_\ell$ is the unit vector whose ℓ-th dimension is one and all other dimensions are zero. In particular, since every coreness vector is dominated by some MCV, there exists an original MCV $\mathbf{h} \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{h} + \mathbf{e}_\ell \succeq \mathbf{k}'$.*

Based on Lemma 1, once such a coreness vector $\mathbf{k}$ is identified, $\mathbf{k} + \mathbf{e}_\ell$ can be used as an upper-bound vector for the corresponding updated MCV. We next show how to identify such affected vectors for single-edge insertion.

Affected scope. For an edge $e = (u, w, \ell)$ inserted on layer ℓ, we maintain an affected vertex set $\mathcal{A}(e)$ and, for each vertex v, an affected vector set $\mathcal{A}_\Phi(e, v)$. Given a vector $\mathbf{k} \in \mathbb{N}^{|L|}$, we define its subspace vector set as $\mathcal{D}(\mathbf{k}) = \{\mathbf{x} \in \mathbb{N}^{|L|} \mid \mathbf{k} \succeq \mathbf{x}\}$. Note that $\mathbf{k} \in \mathcal{D}(\mathbf{k})$. Since every coreness vector is dominated by some MCV, the coreness vectors in Lemma 1 can be identified from the subspaces of the original MCVs. We call such a subspace vector potentially affected if its promotion on layer ℓ may provide an upper bound for a newly generated MCV. The key question is how to identify these affected vectors. Next, we consider two cases.

Affected vectors of inserted endpoints. For a vertex u to belong to a $\mathbf{k}$-core, it must have sufficiently many neighbors that can also satisfy the degree constraints specified by $\mathbf{k}$. Therefore, a neighbor $w \in N_{\mathcal{G}}(u)$ can contribute to maintaining u under $\mathbf{k}$ only if w itself can belong to a $\mathbf{k}$-core. Accordingly, we say that a candidate vector $\mathbf{k}$ of u is *supported* by w if w has an MCV that dominates $\mathbf{k}$. Intuitively, if the insertion of an edge (u, w, ℓ) introduces a new neighbor that supports a coreness vector $\mathbf{k}$ of u, then u may be included in a $(\mathbf{k} + \mathbf{e}_\ell)$-core in the updated graph.

For $\mathbf{k} \in \Phi_{\mathcal{G}}(u)$, $\mathbf{k}' \in \Phi_{\mathcal{G}}(w)$, let $\mathbf{r} = \mathbf{k} \wedge \mathbf{k}' = [\min\{k_{\ell'}, k'_{\ell'}\}]_{\ell' \in L}$. The vector $\mathbf{r}$ is the component-wise greatest vector dominated by both $\mathbf{k}$ and $\mathbf{k}'$. Hence, $\mathbf{r}$ is a coreness vector of u supported by w through $\mathbf{k}'$ and u may be included in a $(\mathbf{r}+\mathbf{e}_\ell)$-core after the insertion of (u, w, ℓ). If $\mathbf{r} + \mathbf{e}_\ell$ is already dominated by an original MCV of u, then any updated MCV dominated by $\mathbf{r} + \mathbf{e}_\ell$ is also dominated by that original MCV and cannot become a new MCV. Otherwise, we include $\mathbf{r}$ in the affected vector set. Therefore, we initialize $\mathcal{A}_\Phi(e, u) = \{\mathbf{r} \mid \exists \mathbf{k} \in \Phi_{\mathcal{G}}(u), \exists \mathbf{k}' \in \Phi_{\mathcal{G}}(w), \mathbf{r} = \mathbf{k} \wedge \mathbf{k}', \nexists \mathbf{h} \in \Phi_{\mathcal{G}}(u) : \mathbf{h} \succeq \mathbf{r} + \mathbf{e}_\ell\}$. Similarly, $\mathcal{A}_\Phi(e, w)$ is initialized symmetrically.

Affected vectors of other vertices. After increasing by one on layer ℓ, an affected vector may newly dominate a promoted subspace vector of a neighboring MCV, thereby causing that vector to become affected as well. This propagation then continues recursively. Next, we formally define this insertion transition relation.

Definition 4. *(Insertion Transition). Given an inserted edge $e = (u, w, \ell)$, a pair $(x, \mathbf{k})$ with $\mathbf{k} \in \mathcal{A}_\Phi(e, x)$ and a pair $(y, \mathbf{k}')$ with $\mathbf{k}' \in \Phi_{\mathcal{G}}(y)$, let $\mathbf{r} = \mathbf{k} \wedge \mathbf{k}'$. There is an insertion transition from $(x, \mathbf{k})$ to $(y, \mathbf{r})$ of e for $x \in N_{\mathcal{G}'}(y)$ if (1) $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{r} + \mathbf{e}_\ell$; (2) $\nexists \mathbf{h} \in \Phi_{\mathcal{G}}(x) : \mathbf{h} \succeq \mathbf{r} + \mathbf{e}_\ell$; and (3) $\nexists \mathbf{q} \in \Phi_{\mathcal{G}}(y) : \mathbf{q} \succeq \mathbf{r} + \mathbf{e}_\ell$.*

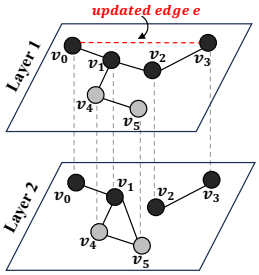
Condition (1) is always satisfied because $\mathbf{r} = \mathbf{k} \wedge \mathbf{k}'$ implies $\mathbf{k} \succeq \mathbf{r}$, and hence $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{r} + \mathbf{e}_\ell$. We include it to explicitly indicate that the transition is triggered by the promoted vector of $\mathbf{k}$. In other words, an affected vector $\mathbf{k}$ may influence a neighbor only when its promoted vector $\mathbf{k} + \mathbf{e}_\ell$ newly dominates a promoted subspace vector $\mathbf{r} + \mathbf{e}_\ell$ derived from one of the neighbor's MCVs $\mathbf{k}'$.

Example 1. *In Figure 2, consider inserting edge $e = (v_0, v_3, 1)$ into $\mathcal{G}$, where the inserted layer is $\ell = 1$. For the MCV $\mathbf{k}_0 = (1, 1)$ of v_0 and the MCV $\mathbf{k}_3 = (1, 1)$ of v_3, we have $\mathbf{r}_0 = \mathbf{k}_0 \wedge \mathbf{k}_3 = (1, 1)$. Since $\mathbf{r}_0 + \mathbf{e}_1 = (2, 1)$ is not dominated by any original MCV of v_0, $(1, 1)$ becomes an affected vector of v_0, indicating that its promoted vector $(2, 1)$ may become an updated MCV, as shown in Figure 2(c). The promotion of this affected vector can further propagate to neighboring vertices. For the MCV $\mathbf{k}_1 = (1, 2)$ of its neighbor v_1, let $\mathbf{r}_1 = \mathbf{r}_0 \wedge \mathbf{k}_1 = (1, 1)$. Then $\mathbf{r}_0 + \mathbf{e}_1 \succeq \mathbf{r}_1 + \mathbf{e}_1 = (2, 1)$. Moreover, neither $(1, 1) \in \Phi_{\mathcal{G}}(v_0)$ nor $(1, 2) \in \Phi_{\mathcal{G}}(v_1)$ dominates $(2, 1)$. Hence, all three conditions in Definition 4 are satisfied, and there is an insertion transition from $(v_0, (1, 1))$ to $(v_1, (1, 1))$. This implies that $(1, 1)$ is an affected vector of v_1, whose promoted vector $(2, 1)$ may become an updated MCV, as shown in Figure 2(c).*

Lemma 2 (Same-level Insertion Transition). *If there is an insertion transition from $(x, \mathbf{k})$ to $(y, \mathbf{r})$ for an edge insertion on layer ℓ, then $k_\ell = r_\ell$.*

The property of Lemma 2 enables faster condition checking for insertion transitions based on k_ℓ and r_ℓ.

Definition 5. *(Insertion Affected Path). Given an inserted edge $e = (u, w, \ell)$, two vector pairs $(x, \mathbf{k})$ and $(y, \mathbf{k}')$, an insertion affected*



(a) A multilayer graph $\mathcal{G}$

v	MCV Set
v_0	(1,1)
v_1	(1,2)
v_2	(1,1)
v_3	(1,1)
v_4	(1,2)
v_5	(1,2)

(b) MCVs of $\mathcal{G}$

v	MCV Set
v_0	(2,1)
v_1	(1,2) (2,1)
v_2	(2,1)
v_3	(2,1)
v_4	(1,2)
v_5	(1,2)

(c) MCVs of $\mathcal{G} \cup \{e\}$

Figure 2: An example of transition

path of e from $(x, \mathbf{k})$ to $(y, \mathbf{k}')$ is a sequence $(x_0, \mathbf{k}_0), \ldots, (x_t, \mathbf{k}_t)$ such that $(x_0, \mathbf{k}_0) = (x, \mathbf{k})$, $(x_t, \mathbf{k}_t) = (y, \mathbf{k}')$, $k_{i,\ell} = k_\ell$ for all $0 \le i \le t$, and there is an insertion transition from $(x_i, \mathbf{k}_i)$ to $(x_{i+1}, \mathbf{k}_{i+1})$.

By Definition 5, $\mathcal{A}_\Phi(e, \cdot)$ can be computed by collecting the affected vectors during the traverse along the insertion affected path. Next, we show how to construct upper-bound vector set $\overline{\Phi}_{\mathcal{G}'}(\cdot)$ based on $\mathcal{A}_\Phi(e, \cdot)$ and $\Phi_{\mathcal{G}}(\cdot)$. Intuitively, by identifying all potentially affected vectors and promoting each of them by $\mathbf{e}_\ell$, we ensure that every newly generated MCV is dominated by at least one promoted vector by Lemma 1.

Lemma 3 (Edge Insertion Affected Scope). *Let $\mathcal{G}' = \mathcal{G} \cup \{e\}$. For any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$, there exists a vector $\overline{\mathbf{k}} \in \Phi_{\mathcal{G}}(v) \cup \{\mathbf{r} + \mathbf{e}_\ell \mid \mathbf{r} \in \mathcal{A}_\Phi(e, v)\}$ such that $\overline{\mathbf{k}} \succeq \mathbf{k}'$.*

Together with the original MCVs, the promoted affected vectors yield a complete upper-bound vector set after dominance filtering.

5.2 Edge Deletion

In this subsection, we discuss the affected bound and affected scope under single-edge deletion. Since each dimension of an MCV must be non-negative, we use $\mathbf{k}^{(\ell,-a)} = \mathbf{k} - \min\{k_\ell, a\} \cdot \mathbf{e}_\ell$ to denote the vector obtained by decreasing the ℓ-th dimension of $\mathbf{k}$ by a, truncated at zero.

Affected bound. After deleting an edge, for every vertex v, each updated MCV in $\Phi_{\mathcal{G}'}(v)$ is dominated by some original MCV in $\Phi_{\mathcal{G}}(v)$. Thus, $\Phi_{\mathcal{G}}(v)$ is a natural upper-bound vector set for deletion. Next, we show that for each original MCV, decreasing one on the deleted layer gives a valid coreness vector in the updated graph.

Lemma 4 (Edge Deletion Bound). *Let $\mathcal{G}' = \mathcal{G} \setminus \{e\}$, where e is deleted from layer ℓ. For any vertex $v \in V$ and any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$ such that $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$.*

By Lemma 4, $\mathbf{k}^{(\ell,-1)}$ is a valid coreness vector of v in the updated graph. If an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, after decreasing its value by one on the updated layer, still dominates an MCV $\mathbf{h}$ of a neighboring vertex, then there must exist an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$ that dominates $\mathbf{h}$, i.e., $\mathbf{k}' \succeq \mathbf{k}^{(\ell,-1)} \succeq \mathbf{h}$. Therefore, the dominance relation from v to $\mathbf{h}$ is preserved in the updated graph $\mathcal{G}'$, and $\mathbf{h}$ cannot be affected by the decrease of $\mathbf{k}$. Next, we discuss how to identify the affected scope for single-edge deletion.

Affected scope. For an edge deletion $e = (u, w, \ell)$, an affected vector refers to an original MCV that may no longer remain an MCV in the updated graph. We discuss the affected vectors for endpoints of deleted edges and other vertices, respectively.

Affected vectors of deleted endpoints. At an endpoint, an original MCV can be directly affected only when the other endpoint of the deleted edge has an MCV dominating it. Thus, the affected vector sets of the endpoints of the deleted edge are initialized as $\mathcal{A}_\Phi(e, u) = \{\mathbf{k} \in \Phi_{\mathcal{G}}(u) \mid \exists \mathbf{h} \in \Phi_{\mathcal{G}}(w), \mathbf{h} \succeq \mathbf{k}\}$ and $\mathcal{A}_\Phi(e, w) = \{\mathbf{k} \in \Phi_{\mathcal{G}}(w) \mid \exists \mathbf{h} \in \Phi_{\mathcal{G}}(u), \mathbf{h} \succeq \mathbf{k}\}$, respectively.

Affected vectors of other vertices. Based on the affected vector sets of the endpoints of the deleted edge, other affected vectors can appear only when the possible loss of a dominance relation is propagated through neighboring vertices.

Definition 6. *(Deletion Transition). Given a deleted edge $e = (u, w, \ell)$, a pair $(x, \mathbf{k})$ with $\mathbf{k} \in \mathcal{A}_\Phi(e, x)$ and a pair $(y, \mathbf{k}')$ with $\mathbf{k}' \in \Phi_{\mathcal{G}}(y)$, there is a deletion transition from $(x, \mathbf{k})$ to $(y, \mathbf{k}')$ of e for $x \in N_{\mathcal{G}'}(y)$ if (1) $\mathbf{k} \succeq \mathbf{k}'$ and (2) $\nexists \mathbf{h} \in \Phi_{\mathcal{G}}(x) : \mathbf{h}^{(\ell,-1)} \succeq \mathbf{k}'$.*

Specifically, when an original MCV $\mathbf{k}$ of a vertex x is affected, it may no longer dominate an MCV $\mathbf{k}'$ of a neighboring vertex y after the update, making $\mathbf{k}'$ a candidate affected vector. However, by Lemma 4, for each $\mathbf{h} \in \Phi_{\mathcal{G}}(x)$, $\mathbf{h}^{(\ell,-1)}$ remains a valid coreness vector of x in the updated graph. Thus, if there exists $\mathbf{h} \in \Phi_{\mathcal{G}}(x)$ such that $\mathbf{h}^{(\ell,-1)} \succeq \mathbf{k}'$, then $\mathbf{k}'$ can still be supported by x and cannot be affected. Therefore, such candidate vectors can be safely pruned.

Example 2. *In Figure 2, consider deleting the inserted edge $e = (v_0, v_3, 1)$ from the updated graph $\mathcal{G} \cup \{e\}$, where the deleted layer is $\ell = 1$. Since the MCV $(2, 1)$ of endpoint v_3 dominates the MCV $(2, 1)$ of endpoint v_0 in $\mathcal{G} \cup \{e\}$, $\mathbf{k} = (2, 1)$ is an affected vector of v_0. For its neighbor v_1, consider $\mathbf{k}' = (2, 1) \in \Phi_{\mathcal{G} \cup \{e\}}(v_1)$. First, $\mathbf{k} \succeq \mathbf{k}'$. Moreover, the only MCV $\mathbf{h} = (2, 1)$ of v_0 satisfies $\mathbf{h}^{(1,-1)} = (1, 1) \not\succeq (2, 1) = \mathbf{k}'$. Thus, after decreasing its value by one on layer 1, no MCV of v_0 can still dominate $\mathbf{k}'$. Therefore, there is a deletion transition from $(v_0, (2, 1))$ to $(v_1, (2, 1))$. Consequently, $(2, 1)$ is identified as an affected vector of v_1, since it may no longer remain a valid MCV after the deletion, as shown in Figure 2(b).*

Lemma 5 (Same-Level Deletion Transition). *If there is a deletion transition from $(x, \mathbf{k})$ to $(y, \mathbf{k}')$ for an edge deletion on layer ℓ, then $k_\ell = k'_\ell$.*

By Lemma 5, candidate affected vectors only need to be checked among MCVs sharing the same value on layer ℓ.

Definition 7. *(Deletion Affected Path). Given a deleted edge $e = (u, w, \ell)$, two vector pairs $(x, \mathbf{k})$ and $(y, \mathbf{k}')$, a deletion affected path of e from $(x, \mathbf{k})$ to $(y, \mathbf{k}')$ is a sequence $(x_0, \mathbf{k}_0), \ldots, (x_t, \mathbf{k}_t)$ such that $(x_0, \mathbf{k}_0) = (x, \mathbf{k})$, $(x_t, \mathbf{k}_t) = (y, \mathbf{k}')$, $\mathbf{k}_i \in \Phi_{\mathcal{G}}(x_i)$, $k_{i,\ell} = k_\ell$ for all $0 \le i \le t$, and there is a deletion transition from $(x_i, \mathbf{k}_i)$ to $(x_{i+1}, \mathbf{k}_{i+1})$.*

$\mathcal{A}_\Phi(e, v)$ can be computed by collecting the affected vectors $\mathbf{k}$ along the deletion affected path of e. The following lemma ensures that it contains all MCVs of v that may need to be updated.

Lemma 6 (Edge Deletion Affected Scope). *Let $\mathcal{G}' = \mathcal{G} \setminus \{e\}$. For any vertex $v \in V$, if an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ may become invalid after deleting e, then $\mathbf{k} \in \mathcal{A}_\Phi(e, v)$.*

After deleting an edge $e = (u, w, \ell)$, we initialize $\mathcal{A}_\Phi(e, u)$ and $\mathcal{A}_\Phi(e, w)$, and insert the endpoints into $\mathcal{A}(e)$ if their affected vector

Algorithm 2: Identify Scope (IS)

Input: Updated graph $\mathcal{G}'$; input MCV sets $\Phi_{\mathcal{G}}(\cdot)$; updated edge $e = (u, w, \ell)$; operation $op \in \{\text{insert}, \text{delete}\}$
Output: Affected vertex set $\mathcal{A}(e)$; upper-bound vector sets $\overline{\Phi}_{\mathcal{G}'}(\cdot)$

1 $\overline{\Phi}_{\mathcal{G}'}(\cdot) \leftarrow \Phi_{\mathcal{G}}(\cdot)$;
2 Compute $\mathcal{A}_\Phi(e, u)$ and $\mathcal{A}_\Phi(e, w)$;
3 **if** op = insert **then**
4 $\quad (\mathcal{A}_\Phi(e, \cdot), \mathcal{A}(e)) \leftarrow$ traverse insertion affected paths of e;
5 **else**
6 $\quad (\mathcal{A}_\Phi(e, \cdot), \mathcal{A}(e)) \leftarrow$ traverse deletion affected paths of e;
7 **if** op = insert **then**
8 $\quad$ **foreach** $v \in \mathcal{A}(e)$ *and* $\mathbf{k} \in \mathcal{A}_\Phi(e, v)$ **do**
9 $\quad\quad$ insert $\mathbf{k} + \mathbf{e}_\ell$ into $\overline{\Phi}_{\mathcal{G}'}(v)$;
10 $\quad\quad$ Filter dominance relations in $\overline{\Phi}_{\mathcal{G}'}(v)$;
11 **return** $(\mathcal{A}(e), \overline{\Phi}_{\mathcal{G}'}(\cdot))$;

sets are nonempty. We then traverse deletion affected paths of the deleted edge e. Whenever an affected vector $\mathbf{k}$ of a vertex v is reached, we add v to $\mathcal{A}(e)$ and insert $\mathbf{k}$ into $\mathcal{A}_\Phi(e, v)$. Finally, we return $\mathcal{A}(e)$ and the upper-bound vector set $\overline{\Phi}_{\mathcal{G}'}(\cdot) = \Phi_{\mathcal{G}}(\cdot)$.

5.3 Identify Affected Scope

Algorithm 2 shows the pseudocode for affected scope identification. It initializes the upper-bound vector sets (line 1), and then computes the affected vector sets of the endpoints of the updated edge (line 2). According to the operation type, it traverses either insertion affected paths or deletion affected paths of e to obtain $\mathcal{A}_\Phi(e, \cdot)$ and $\mathcal{A}(e)$ (lines 3-6). For insertion, every affected vector $\mathbf{k} \in \mathcal{A}_\Phi(e, v)$ is promoted to $\mathbf{k} + \mathbf{e}_\ell$ and inserted into the upper-bound vector set of v, followed by dominance filtering (lines 7-10). For deletion, the upper-bound vector sets remain the original MCV sets as initialized in line 1. Finally, the algorithm returns $\mathcal{A}(e)$ and $\overline{\Phi}_{\mathcal{G}'}(\cdot)$ (line 11).

Correctness analysis. For edge insertion, Lemma 3 guarantees that traversing the insertion affected paths identifies all affected vectors needed to construct an upper-bound vector set that dominates every updated MCV. For edge deletion, Lemma 6 ensures that every original MCV that may become invalid is included in the identified affected vectors, while the original MCV set naturally remains an upper-bound vector set of the updated MCVs. Therefore, Algorithm 2 identifies a complete affected scope and returns valid upper-bound vector sets for both insertion and deletion.

Complexity analysis. Let $P = \max_{v \in V} |\Phi_{\mathcal{G}}(v)|$, $\lambda = |L|$, and $d_v = |N_{\mathcal{G}}(v)|$. Let V_e denote the set of vertices visited during the affected path traversal of the updated edge e. Each visited vertex examines transitions between its affected vectors and the MCVs of its neighboring vertices. Let $P_e = \max_{v \in V_e} |\mathcal{A}_\Phi(e, v)|$ denote the maximum number of affected vectors associated with a visited vertex during traversal triggered by the updated edge e. Therefore, the number of transitions is $O(\sum_{v \in V_e} d_v P_e P)$. For each transition, validating the transition conditions requires dominance checks against at most P MCVs, where each dominance comparison involves λ dimensions, resulting in $O(P\lambda)$ time. Therefore, the complexity of Algorithm 2 is $O(\sum_{v \in V_e} d_v P_e P^2 \lambda)$.

6 LOCAL CONVERGENCE

In this section, we first provide an overview of the local convergence procedure, followed by a naive implementation. We then present several optimization techniques to improve its efficiency.

6.1 Local Convergence Overview

The local convergence computes the MCVs by repeatedly refining invalid upper-bound vectors until all remaining vectors are valid, which is a general procedure applicable to arbitrary multilayer graphs and their corresponding upper-bound vector sets. To maintain generality, we use $\mathcal{G}$ throughout this section to denote the graph currently processed by local convergence, and all neighbor sets, upper-bound vector sets, and MCV sets are defined with respect to $\mathcal{G}$. In Algorithm 1, after obtaining the upper-bound vector sets $\overline{\Phi}_{\mathcal{G}'}(\cdot)$ of the updated graph $\mathcal{G}'$, we invoke local convergence by instantiating $\mathcal{G}$ as $\mathcal{G}'$ to compute the updated MCV sets $\Phi_{\mathcal{G}'}(\cdot)$.

Validation rule. By Definition 3, if $\overline{\mathbf{k}} \in \overline{\Phi}_{\mathcal{G}}(v)$ is an MCV of v, then v must have enough neighbors containing MCVs that dominate $\overline{\mathbf{k}}$. We say that $\overline{\mathbf{k}} = [\overline{k}_\ell]_{\ell \in L}$ is *valid* if $\forall \ell \in L$, there exist at least $\overline{k}_\ell$ neighbors u of v on layer ℓ such that $\exists \overline{\mathbf{k}}' \in \overline{\Phi}_{\mathcal{G}}(u)$ with $\overline{\mathbf{k}}' \succeq \overline{\mathbf{k}}$.

Lemma 7 (Validation Rule). *If every vector $\overline{\mathbf{k}} \in \overline{\Phi}_{\mathcal{G}}(v)$ is valid for any $v \in V$, then $\overline{\mathbf{k}}$ is a coreness vector of v in $\mathcal{G}$.*

For each invalid upper-bound vector, we follow a refinement rule to explore a possible MCV.

Refinement rule. If an upper-bound vector $\overline{\mathbf{k}}$ is invalid, we remove it and replace it with a set of safe refinement vectors $\mathcal{R}(\overline{\mathbf{k}})$. The refinement is *safe* if it satisfies the following two conditions: (1) every $\overline{\mathbf{k}}' \in \mathcal{R}(\overline{\mathbf{k}})$ is strictly dominated by $\overline{\mathbf{k}}$, i.e., $\overline{\mathbf{k}} \succ \overline{\mathbf{k}}'$; (2) for every MCV $\mathbf{k}^* \in \Phi_{\mathcal{G}}(v)$ with $\overline{\mathbf{k}} \succeq \mathbf{k}^*$, if $\overline{\mathbf{k}} \neq \mathbf{k}^*$, then there exists $\overline{\mathbf{k}}' \in \mathcal{R}(\overline{\mathbf{k}})$ such that $\overline{\mathbf{k}}' \succeq \mathbf{k}^*$.

Lemma 8 (Safe Refinement). *Suppose that for every vertex $x \in V$, every MCV of x is dominated by at least one vector in $\overline{\Phi}_{\mathcal{G}}(x)$. Given an invalid vector $\overline{\mathbf{k}} \in \overline{\Phi}_{\mathcal{G}}(v)$ and a safe refinement set $\mathcal{R}(\overline{\mathbf{k}})$, let $\overline{\Phi}'_{\mathcal{G}}(v) = \overline{\Phi}_{\mathcal{G}}(v) \setminus \{\overline{\mathbf{k}}\} \cup \mathcal{R}(\overline{\mathbf{k}})$. Then every MCV of v is dominated by at least one vector in $\overline{\Phi}'_{\mathcal{G}}(v)$.*

Intuitively, the validation rule removes vectors in $\overline{\Phi}_{\mathcal{G}}(v)$ that fail to be an MCV, while the refinement rule ensures the maximality of the retained vectors. As a result, $\overline{\Phi}_{\mathcal{G}}(v)$ iteratively converges to $\Phi_{\mathcal{G}}(v)$. Specifically, the local convergence procedure is shown in Algorithm 3. It first initializes a unique queue with affected vertices in $\mathcal{A}(e)$ (line 1). For each dequeued vertex (line 3), it repeatedly checks whether there exists a vector in $\overline{\Phi}_{\mathcal{G}}(v)$ that is invalid (line 5). Each invalid vector is removed and replaced with a safe refinement set (lines 6-7). Then, dominance filtering is applied to maintain the mutual incomparability of vectors in $\overline{\Phi}_{\mathcal{G}}(v)$ (line 8). If $\overline{\Phi}_{\mathcal{G}}(v)$ changes, only affected neighbors of v are re-enqueued for further validation (lines 10-11). The procedure terminates when the queue becomes empty and returns the remaining upper-bound sets as MCV sets (line 12). Since the validation and refinement of each vertex depend only on the upper-bound vector sets of its neighbors from the current iteration, multiple affected vertices can be processed in parallel within the same iteration.

Algorithm 3: Local Convergence (LC)

Input: A multilayer graph $\mathcal{G}$; upper-bound vector sets $\overline{\Phi}_{\mathcal{G}}(\cdot)$; affected vertex set $\mathcal{A}(e)$
Output: The MCV sets $\Phi_{\mathcal{G}}(\cdot)$

```
Initialize a unique queue Q ← A(e);
while Q ≠ ∅ do
    Dequeue a vertex v from Q;
    changed ← false;
    while ∃ k̄ ∈ Φ̄_G(v) such that k̄ is invalid do
        Remove k̄ from Φ̄_G(v);
        Insert a safe refinement set R(k̄) into Φ̄_G(v);
        Filter dominance relations in Φ̄_G(v);
        changed ← true;
    if changed = true then
        Enqueue all vertices in N_G(v) ∩ A(e) into Q;
return Φ_G(·) ← Φ̄_G(·);
```

Correctness analysis. In Algorithm 3, each refinement strictly decreases at least one dimension of an invalid upper-bound vector while preserving the non-negativity of all dimensions, thereby guaranteeing termination. By Lemma 8, each refinement preserves the invariant that every MCV is dominated by at least one current upper-bound vector. Once such an upper-bound vector becomes a valid coreness vector, it must coincide with the MCV it dominates by the maximality of the MCV. When the queue becomes empty, Lemma 7 ensures that all retained vectors are valid coreness vectors. Therefore, after removing dominated vectors, the remaining vectors are exactly the MCVs.

6.2 Naive Implementation

In this subsection, we introduce a naive implementation of validation and refinement in Algorithm 3.

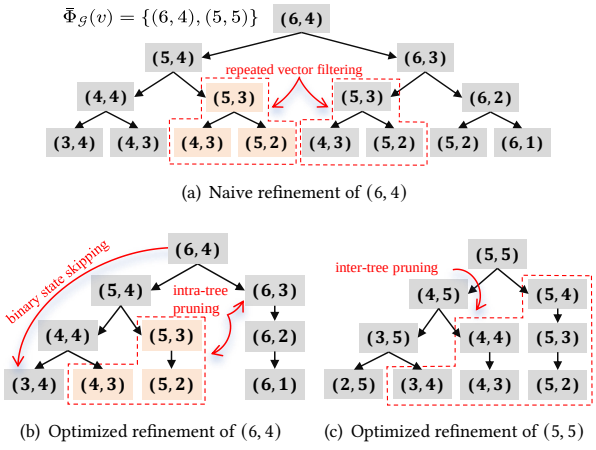
Naive validation. Given a vertex v and a candidate vector $\overline{\mathbf{k}} \in \overline{\Phi}_{\mathcal{G}}(v)$, the naive validation scans $\overline{\Phi}_{\mathcal{G}}(u)$ for each neighbor $u \in N_{\mathcal{G}}(v)$ and checks whether there are enough neighbors in each layer that contain upper-bound vectors dominating $\overline{\mathbf{k}}$.

Naive refinement. For refinement, the naive implementation uses a simple splitting strategy: an invalid vector $\overline{\mathbf{k}}$ is split by decreasing one dimension by one, generating $\{\overline{\mathbf{k}}^{(\ell,-1)} \mid \ell \in L, \overline{k}_\ell > 0\}$, where $\overline{\mathbf{k}}^{(\ell,-1)} = \overline{\mathbf{k}} - \min\{\overline{k}_\ell, 1\} \cdot \mathbf{e}_\ell$. Suppose that $\mathbf{k}^*$ is an MCV dominated by $\overline{\mathbf{k}}$. Since $\overline{\mathbf{k}}$ is invalid, $\overline{\mathbf{k}}$ cannot be an MCV. Therefore, $\overline{\mathbf{k}} \succ \mathbf{k}^*$. This refinement is safe because at least one dimension of $\overline{\mathbf{k}}$ is larger than $\mathbf{k}^*$, and decreasing that dimension by one still yields a vector dominating $\mathbf{k}^*$.

Example 3. *Figure 3(a) is an example of naive refinement. Starting from an invalid upper-bound vector $\overline{\mathbf{k}} = (6,4) \in \overline{\Phi}_{\mathcal{G}}(v)$, the naive refinement generates candidate vectors by decreasing each dimension of $\mathbf{k}$ by one. For example, $(6,4)$ is split into $(5,4)$ and $(6,3)$, which are then recursively validated until maximal valid vectors are identified.*

6.3 Optimized Implementation

Despite its simplicity, the naive implementation incurs significant overhead for two main reasons. First, naive validation repeatedly scans upper-bound vectors of neighbors. Second, naive refinement may generate a large number of invalid intermediate vectors. To address these limitations, we introduce a series of optimizations to accelerate the validation and refinement phase, respectively.



(a) Naive refinement of $(6,4)$

(b) Optimized refinement of $(6,4)$ (c) Optimized refinement of $(5,5)$

Figure 3: An example of refinement

6.3.1 Validation optimization. To reduce redundant accesses to neighboring vertices' upper-bound vectors, we introduce a reuse strategy to leverage historical validation results efficiently.

OPT-1: Reuse strategy. The reuse strategy consists of two operations: *recording* and *updating*. The former records historical validation results of neighbors, while the latter incrementally maintains the recorded information.

Recording. For each upper-bound vector $\overline{\mathbf{k}} \in \overline{\Phi}_{\mathcal{G}}(v)$ and each neighbor $u \in N_{\mathcal{G}}(v)$, we classify the upper-bound vectors in $\overline{\Phi}_{\mathcal{G}}(u)$ into three categories: the dominant set $Dom(\overline{\mathbf{k}}, v, u)$, the fail set $Fail(\overline{\mathbf{k}}, v, u)$, and the candidate set $Cand(\overline{\mathbf{k}}, v, u)$. For any vector $\overline{\mathbf{k}}' \in \overline{\Phi}_{\mathcal{G}}(u)$, if we have verified that $\overline{\mathbf{k}}' \succeq \overline{\mathbf{k}}$, we place it into $Dom(\overline{\mathbf{k}}, v, u)$, and into $Fail(\overline{\mathbf{k}}, v, u)$ otherwise. If $\overline{\mathbf{k}}'$ has not yet been verified, we store it in $Cand(\overline{\mathbf{k}}, v, u)$.

When validating $\overline{\mathbf{k}}$, we first count the neighbors u whose dominant set $Dom(\overline{\mathbf{k}}, v, u)$ is non-empty. If enough neighbors are found, $\overline{\mathbf{k}}$ is valid without further accesses to neighbors' upper-bound vectors. Otherwise, we only need to examine the neighbors with empty dominant sets. For these neighbors, vectors in $Fail(\overline{\mathbf{k}}, v, u)$ can be safely skipped, since they have already been verified and cannot dominate $\overline{\mathbf{k}}$. Thus, we only check the vectors in $Cand(\overline{\mathbf{k}}, v, u)$ to determine whether any of them can dominate $\overline{\mathbf{k}}$, and stop once enough dominating neighbors are found.

Updating. The updating operation aims to update the reuse sets incrementally when $\overline{\Phi}_{\mathcal{G}}(v)$ changes during refinement.

Lemma 9 (Dominance Inheritance). *Consider an upper-bound vector of v refined from $\overline{\mathbf{k}}$ to $\overline{\mathbf{k}}^{new}$. For any neighbor u of v, every $\overline{\mathbf{k}}' \in Dom(\overline{\mathbf{k}}, v, u)$ satisfies $\overline{\mathbf{k}}' \succ \overline{\mathbf{k}}^{new}$.*

By Lemma 9, we incrementally maintain the affected reuse sets. If $\overline{\mathbf{k}}$ is invalid, then for any vector $\overline{\mathbf{k}}^{new}$ in the refinement set of $\overline{\mathbf{k}}$, we directly inherit $Dom(\overline{\mathbf{k}}, v, u)$ as $Dom(\overline{\mathbf{k}}^{new}, v, u)$ because $\overline{\mathbf{k}} \succ \overline{\mathbf{k}}^{new}$ by

refinement rule. However, vectors that fail to dominate $\overline{\mathbf{k}}$ may dominate $\overline{\mathbf{k}}^{new}$. Therefore, we move $Fail(\overline{\mathbf{k}}, v, u)$ to $Cand(\overline{\mathbf{k}}^{new}, v, u)$.

6.3.2 Refinement Optimization. The naive refinement decreases each dimension by one at each step, forming an $|L|$-ary splitting tree. However, this process may generate a large number of redundant intermediate vectors. To improve efficiency, we optimize the refinement phase from three aspects: repeated vector filtering, binary state skipping, and redundant vector pruning.

OPT-2: Repeated vector filtering. In the splitting tree of naive refinement, the same vector can be generated through different sequences of dimension reductions, resulting in redundant exploration. To address this problem, we introduce an ordered splitting rule that prevents repeated vectors from being generated.

Ordered splitting rule. For each splitting vector, we maintain an index $idx(\overline{\mathbf{k}})$ indicating the last decreased layer. The initial vector has $idx(\overline{\mathbf{k}}) = 1$. When splitting a vector with index i, we only allow decreases on layers $j \geq i$. By decreasing the j-th dimension of $\overline{\mathbf{k}}$, each generated child records the index as j. In this way, every root-to-leaf path decreases layers in a non-decreasing order.

Lemma 10 (Repeated Vector Filtering). *The ordered splitting rule generates each vector at most once.*

Example 4. *By applying the ordered splitting rule, the splitting tree shown in Figure 3(b) avoids the repeated vectors such as* $(5, 3)$ *that appeared in Figure 3(a).*

OPT-3: Binary state skipping. The naive refinement strategy explores a splitting tree by sequentially validating candidate vectors along its branches, thereby forming root-to-leaf search paths. Along each branch, the candidate vectors are ordered by dominance, and their validity exhibits hierarchy.

Lemma 11 (Validity Hierarchy). *Consider two upper-bound vectors* $\overline{\mathbf{k}}$ *and* $\overline{\mathbf{k}}'$ *of the same vertex, where* $\overline{\mathbf{k}} \succeq \overline{\mathbf{k}}'$. *If* $\overline{\mathbf{k}}$ *is valid,* $\overline{\mathbf{k}}'$ *is valid. If* $\overline{\mathbf{k}}'$ *is invalid,* $\overline{\mathbf{k}}$ *is invalid.*

We exploit this property to apply binary search and locate the maximal valid vector on each branch, reducing the number of required validations from linear to logarithmic in the length of the corresponding search range.

Example 5. *In Figure 3(b), suppose that* $(3, 4)$ *is an invalid vector. Therefore, any vector dominating* $(3, 4)$*, such as* $(4, 4)$ *and* $(5, 4)$*, must also be invalid. Under the naive refinement strategy, these vectors are validated sequentially in descending order, requiring validations of* $(5, 4)$ *and* $(4, 4)$ *before checking* $(3, 4)$*. In contrast, OPT-3 applies binary search along each branch. For example, by fixing the value of the second dimension, the search range of the first dimension for* $(6, 4)$ *is initialized as* $[0, 5]$*. The midpoint is selected as* $\lceil (0 + 5)/2 \rceil = 3$*, and OPT-3 first validates* $(3, 4)$*. Since* $(3, 4)$ *is invalid, all vectors dominating* $(3, 4)$ *can be safely pruned by Lemma 11, and the search on the current branch continues only in the remaining range* $[0, 2]$ *to find the maximal valid vector. Therefore, OPT-3 avoids the validations of* $(5, 4)$ *and* $(4, 4)$*. Once the maximal valid vector on the current branch is identified or no valid vector exists, OPT-3 terminates the search on this branch and proceeds to another unexplored branch of the splitting tree, such as the unexplored second branch under* $(4, 4)$*,* $(5, 4)$*, and* $(6, 4)$*. The same procedure is then applied to locate the maximal valid vector on each remaining branch until all branches have been examined.*

OPT-4: Redundant vector pruning. The redundant vector pruning strategy is designed for two scenarios: intra-tree pruning and inter-tree pruning.

Intra-tree pruning. The key idea of the intra-tree pruning is based on the observation that once a vector is dominated by a discovered valid vector of the same vertex, neither this vector nor any of its descendants can become maximal. Therefore, we maintain a maximal valid vector set $Val(v)$ for each vertex to prune these vectors. To fully exploit the maximal valid vector set $Val(v)$, maximal valid vectors should be discovered as early as possible. To this end, we propose a right-subtree-first traversal strategy.

Lemma 12 (Subtree Dominance). *Consider a vector* $\overline{\mathbf{k}}$ *in a splitting tree of the refinement phase and two child subtrees* T_i *and* T_j *generated by decreasing the value of* $\overline{\mathbf{k}}$ *on layers* i *and* j *with* $i < j$*, respectively. Then no vector in* T_i *dominates any vector in* T_j*. In contrast, a vector in* T_j *may dominate a vector in* T_i*.*

By Lemma 12, we traverse right subtrees first so that maximal valid vectors can be discovered earlier to prune the subsequent vectors that are dominated by them.

Example 6. *As illustrated in Figure 3(b), we first explore the right subtree of* $(6, 4)$*. After identifying* $(6, 3)$ *as a maximal valid vector, we insert it into* $Val(v)$*. Subsequently, when traversing the left subtree, we reach* $(5, 3)$*. Since* $(6, 3) \succeq (5, 3)$*,* $(5, 3)$ *is dominated by an existing maximal valid vector. Moreover, every descendant of* $(5, 3)$ *is also dominated by* $(6, 3)$*. Hence, none of them can become maximal valid vectors, and the subtree rooted at* $(5, 3)$ *can be pruned safely.*

Inter-tree pruning. A vertex may have multiple invalid vectors, whose split descendant vectors are validated against the same neighbors. Although these invalid vectors are mutually incomparable, their descendant subspaces may overlap. For a root vector $\overline{\mathbf{r}} \in \mathbb{N}^{|L|}$, let $\mathcal{D}(\overline{\mathbf{r}}) = \{\mathbf{x} \in \mathbb{N}^{|L|} \mid \overline{\mathbf{r}} \succeq \mathbf{x}\}$ denote its subspace vector set.

Lemma 13 (Subspace Coverage). *If the splitting tree rooted at* $\overline{\mathbf{r}}$ *has been fully explored, then no vector in* $\mathcal{D}(\overline{\mathbf{r}})$ *can produce a new maximal valid vector.*

Example 7. *In Figure 3, suppose that* $\overline{\Phi}_{\mathcal{G}}(v) = \{(6, 4), (5, 5)\}$*, where both vectors are invalid upper-bound vectors. After completely refining* $(6, 4)$*, all maximal valid vectors contained in* $\mathcal{D}((6, 4))$ *have been identified. Subsequently, when refining* $(5, 5)$*, as shown in Figure 3(c), the vector* $(5, 4)$ *is generated. Since* $(5, 4) \in \mathcal{D}((6, 4))$*, any maximal valid vector reachable from the subtree rooted at* $(5, 4)$ *has already been discovered during the refinement of* $(6, 4)$ *or dominated by a previously discovered maximal valid vector. Hence, the subtree rooted at* $(5, 4)$ *can be safely pruned.*

Complexity analysis. Let $\lambda = |L|$, $d_v = |N_{\mathcal{G}}(v)|$, and $K_v = \max_{\overline{\mathbf{k}} \in \overline{\Phi}_{\mathcal{G}}(v), \ell \in L}(\overline{k}_\ell + 1)$. Let $\overline{P}$ denote the maximum number of upper-bound vectors of each vertex during the process. Let $\mathcal{T}_v$ denote the number of search branches explored in a single refinement of vertex v. Since binary state skipping reduces the number of validations along each search branch from $O(K_v)$ to $O(\log K_v)$, at most $O(\overline{P}\mathcal{T}_v \log K_v)$ vectors are explored in one refinement of v. For each

explored vector, validation examines the dominance of the neighbors' upper-bound vectors, incurring $O(d_v\overline{P}\lambda)$ time. Let $\mathcal{I}_v$ denote the number of times vertex v is refined during the local convergence. Therefore, Algorithm 3 takes $O\left(\sum_{v\in\mathcal{A}(e)} \mathcal{I}_v\overline{P}^2\mathcal{T}_v \log K_v\, d_v\lambda\right)$ time.

Discussion. The key insight of MCM is that maintaining global decomposition results can be accomplished through a sequence of local identification and update operations, thereby achieving better efficiency. Based on the preceding analysis, the complexity of MCM is $O\left(\sum_{v\in V_e} d_vP_eP^2\lambda + \sum_{v\in\mathcal{A}(e)} \mathcal{I}_v\overline{P}^2\mathcal{T}_v \log K_v\, d_v\lambda\right)$. In contrast, the complexity of the state-of-the-art method [34] is $O\left(\prod_{\ell=1}^{\lambda-1}\kappa(\mathcal{G}_\ell)(\lambda|V|+|E|)\right)$, where $\kappa(\mathcal{G}_\ell)$ denotes the degeneracy of $\mathcal{G}_\ell$. Since it depends on the whole graph size, whereas MCM focuses computation on the locally affected region, MCM is particularly effective for local updates. Overall, MCM offers a novel perspective on multilayer core computation in dynamic settings by exploiting update locality and establishes a theoretical and algorithmic foundation for efficient maintenance through a series of specialized optimizations.

7 EXTENSION TO BATCH UPDATES

In this section, we extend our multilayer core maintenance algorithm to support batch edge updates ΔE.

Hybrid edge grouping. The key idea is to partition ΔE into edge groups such that updating all edges in the same group simultaneously still preserves the single-edge update bound of Lemma 1 and Lemma 4. Therefore, the edges within each group can be processed simultaneously by merging their affected scopes. Next, we introduce our hybrid edge grouping strategy based on two types of edge groups, i.e., *disjoint edge group* and *joint edge group*.

Disjoint edge group. Let updated edge set $\Delta E = \{\Delta E_\ell\}_{\ell\in L}$, where ΔE_ℓ is the updated edges on layer ℓ. An edge set $B \subseteq \Delta E_\ell$ is called a disjoint edge group if no two edges in B share an endpoint. Intuitively, after updating all edges in B, each vertex in a $\mathbf{k}$-core gains or loses at most one incident edge on layer ℓ. Based on this observation, we have the following lemma.

Lemma 14 (Disjoint Edge Group Bound). *Let $B \subseteq \Delta E_\ell$ be a disjoint edge group updated on layer ℓ. If B is inserted, then for any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}\cup B}(v)$, there exists a coreness vector $\mathbf{k}$ of v in $\mathcal{G}$ such that $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$. If B is deleted, then for any vertex $v \in V$ and any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}\setminus B}(v)$ such that $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$.*

By Lemma 14, updating all edges in a disjoint edge group simultaneously preserves the single-edge update bound. Therefore, the entire group can be processed as a single update by merging the affected scope of each edge.

Joint edge group. Despite the effectiveness of disjoint edge groups, they may degenerate when a large number of updated edges share a common endpoint. To address this issue, we introduce a concept of the joint edge group to process these edges simultaneously. Let $\mu_\ell(v) = \min_{\mathbf{k}\in\Phi_{\mathcal{G}}(v)} k_\ell$ and $\eta_\ell(v) = \max_{\mathbf{k}\in\Phi_{\mathcal{G}}(v)} k_\ell$ as the minimum and maximum layer-ℓ values among the original MCVs of v, respectively. Formally, we define the joint edge group as follows.

Definition 8. *(Joint Edge Group). Let $B \subseteq \Delta E_\ell$ be a set of updated edges on layer ℓ and C_B be the set of common endpoints in B. B is a joint edge group if (1) no edge in B connects two vertices in C_B; and (2) for every edge $(u, w, \ell) \in B$ with an endpoint in C_B, $\mu_\ell(u) > \eta_\ell(w)$ if $u \in C_B$, and $\mu_\ell(w) > \eta_\ell(u)$ if $w \in C_B$.*

Next, we show that the update of edges in each joint edge group still preserves the single-edge update bound.

Lemma 15 (Joint Edge Group Bound). *Let $B \subseteq \Delta E_\ell$ be a joint edge group. If B is inserted, then for any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}\cup B}(v)$, there exists a coreness vector $\mathbf{k}$ of v in $\mathcal{G}$ such that $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$. If B is deleted, then for any vertex $v \in V$ and any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}\setminus B}(v)$ such that $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$.*

Intuitively, if the value of every MCV of the common endpoint is greater than those of the non-common endpoints on the updated layer, then this common endpoint cannot be affected according to the same-level requirement of Lemma 2 and Lemma 5. Moreover, each non-common endpoint is incident to at most one updated edge on layer ℓ in B. Therefore, updating all edges in the joint edge group satisfies the single-edge update bound.

Batch updates. By combining disjoint and joint edge groups, our hybrid grouping strategy greedily constructs edge groups from ΔE_ℓ. Specifically, for each selected edge, we insert it into an existing group if either it shares no endpoint with the edges already in that group, or the resulting enlarged group still satisfies the conditions of the joint edge group. Based on the hybrid grouping strategy, for each edge group B, Lemmas 14 and 15 guarantee that updating B preserves the single-edge update bound. We thus traverse the affected paths rooted at edges in B on the updated graph, merge the resulting affected scopes and candidate upper-bound vectors, and finally apply Algorithm 3 to refine these candidates into the updated MCVs. The details can be found in the appendix of [58].

Table 1: Dataset statistics

Dataset	$\|V\|$	$\|E\|$	$\|L\|$	$\min_{\ell\in L}\|E_\ell\|$	$\max_{\ell\in L}\|E_\ell\|$
Askubuntu(AU) [1]	159,316	461,248	5	13	147,120
Superuser(SU) [47]	194,085	730,988	6	37,305	158,762
Yago(YG) [9]	123,183	1,072,765	37	17	377,139
Obama(OB) [39]	2,279,535	3,827,964	3	557,445	1,807,134
Pope(PO) [42]	2,064,866	5,492,251	3	423,103	3,092,745
Boston(BT) [42]	4,374,479	9,130,543	3	665,890	5,909,061

8 EXPERIMENTS

In this section, we conduct experiments to evaluate our maintenance algorithms on real-world multilayer graphs.

8.1 Experimental Setup

Datasets. We evaluate our methods on six real-world multilayer graph datasets from diverse domains, including online interaction networks, knowledge graphs, and social networks. Their detailed statistics are reported in Table 1.

Competitors. We compare several algorithms in our experiments:

- **Dec** [15]: the classic multilayer core decomposition algorithm.
- **ParDec** [34]: the state-of-the-art parallel multilayer core decomposition algorithm.
- **MCM**: our multilayer core maintenance algorithm.

Table 2: Overall performance comparison (ms)

Operation	Dataset	Dec	ParDec	MCM	Speedup	IS	LC
Insertion	AU	11,115	89	2.629	×34	2.260	0.369
	SU	102,263	643	22.815	×28	21.219	1.595
	YG	27,869	2,704	0.485	×5,576	0.297	0.188
	OB	12,874	1,763	2.982	×591	2.044	0.938
	PO	62,330	611	3.527	×173	2.249	1.277
	BT	194,159	7,190	9.732	×739	7.265	2.467
Deletion	AU	11,114	89	0.477	×186	0.386	0.092
	SU	102,213	645	4.661	×138	3.959	0.703
	YG	27,815	2,727	1.450	×1,881	0.910	0.539
	OB	12,672	1,758	3.039	×579	2.018	1.021
	PO	62,312	618	2.397	×258	1.614	0.784
	BT	194,178	7,117	5.581	×1,275	3.708	1.873

- **IS, LC**: the affected scope identification and local convergence procedures of MCM, respectively.
- **MCM-S**: a batch-update variant that processes updated edges sequentially by applying MCM to each edge individually.

Setup. The algorithms are implemented in C++ with `-O3` compiler optimization and OpenMP-based parallelization. Experiments are conducted on a Linux machine equipped with an Intel Xeon Gold 5520 2.2 GHz processor and 125 GB RAM. Following [6], we randomly generate 500 single-edge updates. For batch-edge updates, we generate 500 update batches, each containing 500 updated edges. We report the average running time per update in milliseconds (ms). By default, we conduct the experiments under the single-edge update, and the number of threads is set to 32. For each experiment, we verify the exactness of MCM by comparing the results maintained by MCM with those produced by Dec and ParDec. MCM produces identical results in all experiments, confirming its exactness. Due to space limitations, results on some datasets can be found in the appendix of [58].

8.2 Experimental Results

Exp-1: Overall performance comparison. We compare MCM against Dec and ParDec under the default experimental settings. As shown in Table 2, compared with the state-of-the-art method ParDec, MCM achieves speedups ranging from ×28 to ×5,576 for insertions and from ×138 to ×1,881 for deletions. These results demonstrate that decomposition-based methods are inefficient for dynamic updates, whereas MCM can maintain MCVs efficiently by restricting computation to the affected scope. We also observe that edge insertions generally incur higher overhead than deletions. This is because, for insertion, the subspace vectors dominated by the original MCVs may become affected vectors, whereas for deletion, only the original MCVs themselves can become affected. As a result, insertion operations generate far more affected vectors to process. The runtime breakdown of MCM further shows that IS usually accounts for the major part of the running time under the single-edge update. This is because IS needs to traverse the graph and MCVs to identify the affected scope. In contrast, LC operates only within the compact affected scope.

Exp-2: Effect of optimizations. In this experiment, we evaluate the effectiveness of the optimizations proposed for LC in Section 6.3. We use FULL to denote LC equipped with all optimizations, and -OPT1 to -OPT4 to denote variants where each optimization is disabled individually. Table 3 shows the results. For insertions, removing OPT1–OPT4 results in average performance degradations of 2.73%, 78.02%, 6.28%, and 55.70%, respectively. For deletions, the corresponding degradations are 12.87%, 22.97%, 51.95%, and 50.54%, respectively. However, disabling optimizations may occasionally lead to slightly better performance on some datasets. This is because the affected scope and the number of changed MCVs are typically small under the single-edge update, leaving little redundant validation and refinement to prune in these cases. Consequently, the overhead of checking optimization conditions may outweigh the pruning benefits, although the resulting slowdowns are marginal. In cases with more redundant validation and refinement, the optimizations yield more noticeable performance gains. The performance gains averaged across datasets demonstrate the overall effectiveness of the proposed optimizations.

Table 3: Effect of optimizations (ms)

Operation	Dataset	FULL	-OPT1	-OPT2	-OPT3	-OPT4
Insertion	AU	0.369	0.355	0.362	0.376	0.561
	SU	1.595	1.589	2.581	1.604	2.294
	YG	0.188	0.190	0.953	0.258	0.258
	OB	0.938	0.941	0.945	0.937	1.712
	PO	1.277	1.272	1.287	1.259	2.008
	BT	2.467	2.950	2.461	2.455	3.980
	Avg. +/-	–	-2.73%	-78.02%	-6.28%	-55.70%
Deletion	AU	0.089	0.090	0.190	0.292	0.142
	SU	0.703	0.692	0.736	0.762	0.752
	YG	0.539	0.663	0.500	0.616	0.551
	OB	1.021	1.319	1.025	1.517	1.769
	PO	0.784	0.979	0.893	0.781	1.448
	BT	1.873	1.884	2.109	2.111	3.307
	Avg. +/-	–	-12.87%	-22.97%	-51.95%	-50.54%

Exp-3: Effect of graph size. To evaluate the impact of graph size, we construct induced subgraphs of different sizes by randomly sampling varying percentages of vertices, ranging from 20% to 100%. The results are shown in Figure 4. Overall, the running time of all algorithms increases steadily as the graph size grows. For Dec and ParDec, both methods perform multilayer core decomposition on the entire updated graph, making their computational overhead highly sensitive to the graph scale. For MCM, the running time increases moderately because larger graphs tend to induce a broader candidate affected scope. Nevertheless, MCM consistently outperforms Dec and ParDec across all graph sizes, demonstrating its scalability for dynamic multilayer core maintenance.

Exp-4: Effect of the number of layers. To evaluate the effect of the number of layers, we vary the number of selected layers and extract the induced subgraph for each layer subset. As shown in Figure 5, MCM consistently outperforms Dec and ParDec across all layer configurations. For Dec and ParDec, the multilayer core decomposition becomes more computationally expensive on the multilayer graph with richer layer structures. In contrast, the computational cost of MCM is jointly determined by the affected scope and the cost of updating MCVs within that scope. Although increasing the number of layers raises the dimensionality and complexity of maintenance, the average affected scope under single-edge updates typically remains small relative to the entire graph. Therefore, MCM maintains good scalability w.r.t. the number of layers.

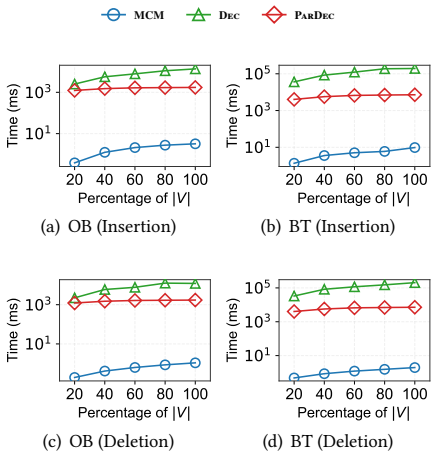


Figure 4: Effect of $|V|$

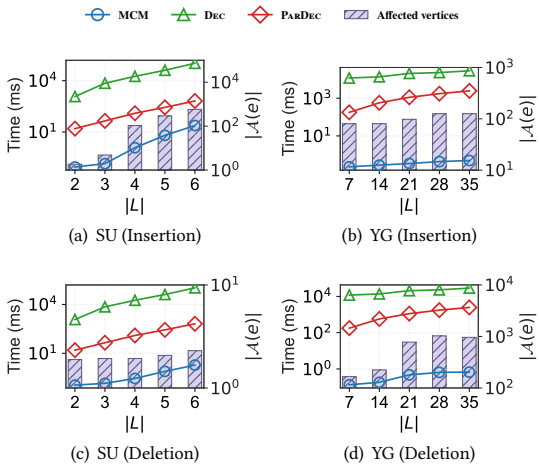


Figure 5: Effect of $|L|$

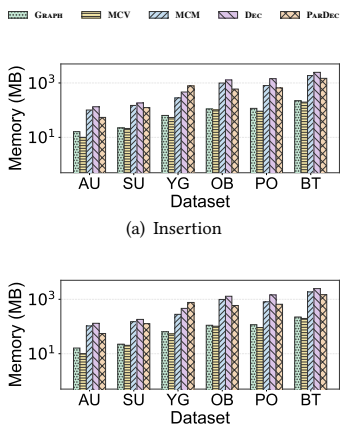


Figure 6: Memory analysis

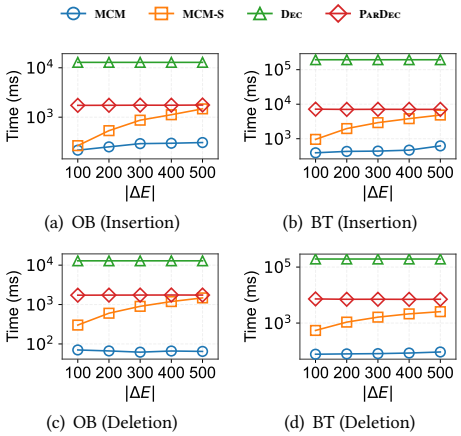
Exp-5: Memory analysis. In this experiment, we analyze the memory usage under edge insertion and deletion separately. Figure 6 shows the results, where Graph and MCV denote the memory usage of the graph and updated MCVs, respectively. The memory required to store the MCVs is slightly lower than that required to store the graph, even on the YG dataset with 37 layers. This is because MCVs are pairwise incomparable, and thus the actual number of MCVs for each vertex is typically much smaller than the theoretical worst-case number of possible vectors. For each method, the memory usage remains nearly identical between edge insertions and deletions due to the limited changes of single-edge updates to the graph structure and the maintained MCVs. Overall, MCM incurs only moderate memory overhead compared with Graph and consistently consumes less memory than Dec, since it performs maintenance only within the affected scope. Compared with ParDec, MCM exhibits slightly higher memory usage on some datasets because it needs to preload the original MCVs of all vertices for local updates during the maintenance process. However, MCM achieves better efficiency than ParDec while incurring only slight additional memory overhead.

Exp-6: Effect of batch updates. We vary the number of updated edges $|\Delta E|$ from 100 to 500. For a fair batch update comparison, all edges in ΔE are first applied to the graph for both Dec and ParDec, after which each method performs decomposition only once on the resulting updated graph, rather than recomputing after every individual edge update. As shown in Figure 7, the running times of Dec and ParDec remain nearly unchanged as $|\Delta E|$ increases. This is because both methods perform decomposition on the entire graph and $|\Delta E| \ll |E|$. Hence, their costs are dominated by the graph size rather than $|\Delta E|$. In contrast, the running time of MCM-S increases steadily with $|\Delta E|$ as it processes the updated edges sequentially. MCM maintains relatively stable performance across different values of $|\Delta E|$ and outperforms MCM-S. The reason is that MCM employs a hybrid edge grouping strategy to support group-wise maintenance, allowing edges within the same group to be processed in parallel. These results demonstrate the effectiveness of the hybrid edge grouping strategy for batch updates.

Exp-7: Effect of the number of threads. We evaluate the performance of MCM w.r.t. the number of threads. Figure 8 reports the results, where MCM-100 to MCM-500 denote MCM with $|\Delta E|$ ranging from 100 to 500. The results show that the running time of MCM decreases as the number of threads increases. Moreover, MCM typically achieves faster speedup with larger batch sizes for edge insertion, since more updated edges can be processed simultaneously. For edge deletion, the running time may occasionally fluctuate or change only slightly because most deleted edges can be processed efficiently, in which case the overhead of using more threads may outweigh the parallelization benefit. Overall, these results demonstrate that MCM can effectively leverage threads to accelerate the multilayer core maintenance.

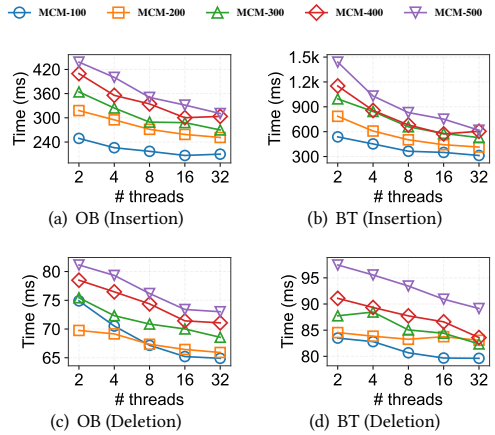
Exp-8: Effect of the distribution of updated edges. We generate 500 edges for each update, and the number of updated edges on layer ℓ is set to $500 \cdot \theta_\ell$. We consider four layer sampling strategies. Let $\text{rank}(\ell)$ denote the rank of layer ℓ in descending order of $|E_\ell|$, where a smaller rank indicates a denser layer. Specifically, UN samples edges uniformly across layers with $\theta_\ell = \frac{1}{|L|}$, SP favors sparser layers with $\theta_\ell = \frac{\text{rank}(\ell)}{\sum_{\ell' \in L} \text{rank}(\ell')}$, DE favors denser layers with $\theta_\ell = \frac{|L| - \text{rank}(\ell) + 1}{\sum_{\ell' \in L} (|L| - \text{rank}(\ell') + 1)}$, and PR samples edges proportionally with $\theta_\ell = \frac{|E_\ell|}{\sum_{\ell' \in L} |E_{\ell'}|}$. As shown in Figure 9, MCM exhibits similar running times across different sampling strategies for most datasets, demonstrating its stability under different distributions of updated edges.

Exp-9: Effect of hybrid edge grouping strategy. We compare MCM with three variants: MCM w/o D, which disables disjoint edges in a group; MCM w/o J, which disables joint edges in a group; and MCM-S, which processes all updated edges sequentially without edge grouping. To vary the concentration of updated edges in a batch, we randomly select a vertex and a layer, and set $s \in \{2, 4, 8, 16, 32\}$. For insertion, we generate s new edges incident to the selected vertex on the selected layer; for deletion, we select s existing edges incident to the selected vertex on that layer. We repeat this process for 100 vertices and apply all selected edges as a batch update. As shown in Figure 10, MCM-S is consistently slower



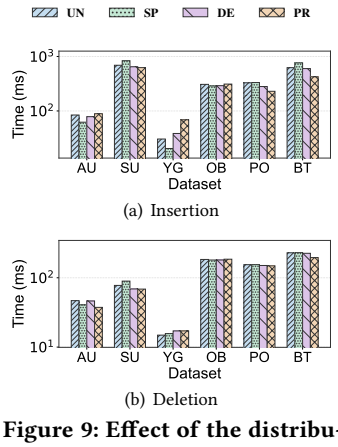
(a) OB (Insertion) (b) BT (Insertion)
(c) OB (Deletion) (d) BT (Deletion)

Figure 7: Effect of $|\Delta E|$



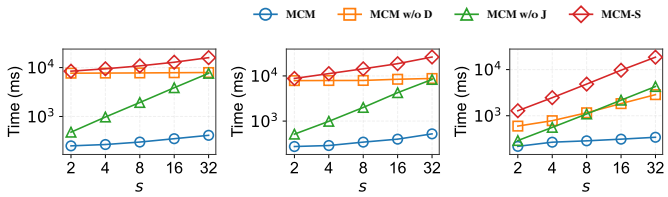
(a) OB (Insertion) (b) BT (Insertion)
(c) OB (Deletion) (d) BT (Deletion)

Figure 8: Effect of # threads



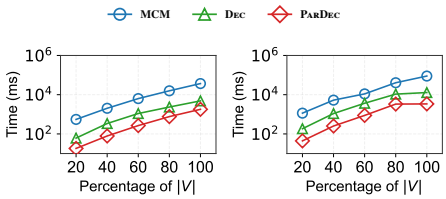
(a) Insertion
(b) Deletion

Figure 9: Effect of the distribution of updated edges



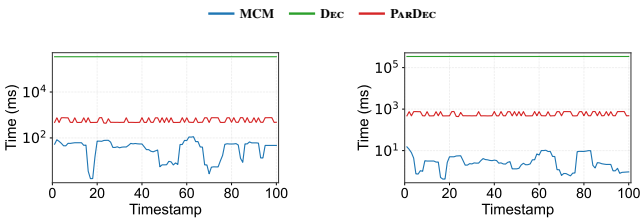
(a) OB (Insertion) (b) BT (Insertion) (c) OB (Deletion) (d) BT (Deletion)

Figure 10: Effect of hybrid edge grouping strategy



(a) OB (b) BT

Figure 11: Full-graph update time



(a) Insertion (b) Deletion

Figure 12: Case study on the temporal network

than the other methods, while MCM, which employs both grouping strategies, consistently achieves the best performance. This demonstrates the effectiveness of both disjoint and joint edge grouping. As s increases, more updated edges share a common endpoint, reducing the opportunities for disjoint edge grouping while allowing more edges to be processed together through joint edge grouping. Therefore, as s increases, the running time of MCM w/o D grows more slowly than that of MCM w/o J.

Exp-10: Performance of full-graph updates. We compare the performance of different methods under full-graph updates by starting from an empty graph and inserting all edges of subgraphs generated at different percentages of the original graph. As shown in Figure 11, MCM is slower than Dec and ParDec in the full-graph update scenario. This is expected because Dec and ParDec are specifically designed for static full-graph decomposition, whereas the main advantage of MCM lies in exploiting update locality and restricting computation to the affected scope. Under full-graph updates, however, the affected scope covers the entire graph, diminishing this advantage while introducing additional maintenance overhead. These results indicate that static decomposition methods are more suitable for full-graph computation, while MCM is better suited to local updates in dynamic multilayer graphs.

Exp-11: Case study. We conduct a case study on the temporal network *MathOverflow* [41], where each vertex represents a user and each edge captures a temporal interaction. Following [6, 32], we sort all temporal edges by their earliest timestamps and divide the entire time span into five consecutive intervals, each of which is regarded as one layer in the multilayer graph. To simulate real network dynamics, we select the last 20 timestamps from each layer, obtaining 100 timestamps in total. The initial multilayer graph is constructed by excluding the edges associated with these selected timestamps. We then incrementally insert and delete the selected edges and report the average update time per timestamp. As shown in Figure 12, MCM consistently outperforms Dec and ParDec across all timestamps, demonstrating that MCM can efficiently support continuous updates in the real-world temporal network.

9 CONCLUSIONS

In this paper, we study the multilayer core maintenance problem on dynamic multilayer graphs. To solve this problem, we propose efficient maintenance algorithms that identify the affected scope for edge updates and compute MCVs via a local convergence procedure with a series of optimization techniques. In addition, we extend the proposed algorithms to batch updates with a hybrid edge grouping strategy. Extensive experiments demonstrate the efficiency of our proposed algorithms. In the future, we would like to investigate the distributed multilayer core maintenance problem for large dynamic multilayer graphs.

REFERENCES

[1] Askubuntu Network. 2026. https://snap.stanford.edu/data/sx-askubuntu.html.

[2] N Azimi-Tafreshi, J Gómez-Gardenes, and SN Dorogovtsev. 2014. k-Core Percolation on Multiplex Networks. *Phys. Rev. E* 90, 3 (2014), 032816.

[3] Wen Bai, Yadi Chen, and Di Wu. 2020. Efficient temporal core maintenance of massive graphs. *Inf. Sci.* 513 (2020), 324–340.

[4] Wen Bai, Yadi Chen, Di Wu, Zhichuan Huang, Yipeng Zhou, and Chuan Xu. 2022. Generalized core maintenance of dynamic bipartite graphs. *Data Min. Knowl. Discov.* 36, 1 (2022), 209–239.

[5] Ali Behrouz, Farnoosh Hashemi, and Laks V. S. Lakshmanan. 2022. FirmTruss Community Search in Multilayer Networks. *Proc. VLDB Endow.* 16, 3 (2022), 505–518.

[6] Yu Chen, Qing Liu, Yifan Zhu, and Yunjun Gao. 2025. Efficient η-Threshold Maintenance in Dynamic Uncertain Graphs. In *ICDE*. 515–528.

[7] Wanyun Cui, Yanghua Xiao, Haixun Wang, and Wei Wang. 2014. Local search of communities in large graphs. In *SIGMOD Conference*. 991–1002.

[8] Jie Dai, Yixiao Liu, Yuan Li, Fazal Wahab, and Yuhai Zhao. 2025. I/O efficient core maintenance in large graphs. *Expert Syst. Appl.* 288 (2025), 128234.

[9] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. 2018. Convolutional 2D Knowledge Graph Embeddings. In *AAAI*. 1811–1818.

[10] Xiangfei Fang, Chengying Huan, Heng Zhang, Yongchao Liu, Shaonan Ma, Yanjun Wu, and Chen Zhao. 2025. *OTM*: Efficient k-Order-Based Core Maintenance in Large-Scale Dynamic Hypergraphs. *ACM Trans. Knowl. Discov. Data* 19, 3 (2025), 79:1–79:33.

[11] Yixiang Fang, Xin Huang, Lu Qin, Ying Zhang, Wenjie Zhang, Reynold Cheng, and Xuemin Lin. 2020. A survey of community search over big graphs. *VLDB J.* 29, 1 (2020), 353–392.

[12] Yixiang Fang, Zhongran Wang, Reynold Cheng, Hongzhi Wang, and Jiafeng Hu. 2019. Effective and Efficient Community Search Over Large Directed Graphs. *IEEE Trans. Knowl. Data Eng.* 31, 11 (2019), 2093–2107.

[13] Yixiang Fang, Yixing Yang, Wenjie Zhang, Xuemin Lin, and Xin Cao. 2020. Effective and Efficient Community Search over Large Heterogeneous Information Networks. *Proc. VLDB Endow.* 13, 6 (2020), 854–867.

[14] FF-TW Network. 2026. https://uuinfolab.github.io/data/.

[15] Edoardo Galimberti, Francesco Bonchi, Francesco Gullo, and Tommaso Lanciano. 2020. Core Decomposition in Multilayer Networks: Theory, Algorithms, and Applications. *ACM Trans. Knowl. Discov. Data* 14, 1 (2020), 11:1–11:40.

[16] Sen Gao, Hongchao Qin, Rong-Hua Li, and Bingsheng He. 2023. Parallel Colorful h-star Core Maintenance in Dynamic Graphs. *Proc. VLDB Endow.* 16, 10 (2023), 2538–2550.

[17] Bin Guo and Emil Sekerinski. 2024. Simplified algorithms for order-based core maintenance. *J. Supercomput.* 80, 13 (2024), 19592–19623.

[18] Yucan Guo, Chenhao Ma, and Yixiang Fang. 2024. Efficient Core Decomposition Over Large Heterogeneous Information Networks. In *ICDE*. 2393–2406.

[19] Farnoosh Hashemi and Ali Behrouz. 2024. A Unified Core Structure in Multiplex Networks: From Finding the Densest Subgraph to Modeling User Engagement. In *KDD*. 1028–1039.

[20] Farnoosh Hashemi, Ali Behrouz, and Laks V. S. Lakshmanan. 2022. FirmCore Decomposition of Multilayer Networks. In *WWW*. 1589–1600.

[21] Qiang-Sheng Hua, Yuliang Shi, Dongxiao Yu, Hai Jin, Jiguo Yu, Zhipeng Cai, Xiuzhen Cheng, and Hanhua Chen. 2020. Faster Parallel Core Maintenance Algorithms in Dynamic Graphs. *IEEE Trans. Parallel Distributed Syst.* 31, 6 (2020), 1287–1300.

[22] Qiang-Sheng Hua, Hongen Wang, Hai Jin, and Xuanhua Shi. 2024. Core Maintenance on Dynamic Graphs: A Distributed Approach Built on H-Index. *IEEE Trans. Big Data* 10, 5 (2024), 595–608.

[23] Qiang-Sheng Hua, Xiaohui Zhang, Hai Jin, and Hong Huang. 2023. Revisiting Core Maintenance for Dynamic Hypergraphs. *IEEE Trans. Parallel Distributed Syst.* 34, 3 (2023), 981–994.

[24] Hongxuan Huang, Qingyuan Linghu, Fan Zhang, Dian Ouyang, and Shiyu Yang. 2021. Truss Decomposition on Multilayer Graphs. In *IEEE BigData*. 5912–5915.

[25] Xin Huang, Laks V. S. Lakshmanan, and Jianliang Xu. 2017. Community Search over Big Graphs: Models, Algorithms, and Opportunities. In *ICDE*. 1451–1454.

[26] Daxin Jiang and Jian Pei. 2009. Mining frequent cross-graph quasi-cliques. *ACM Trans. Knowl. Discov. Data* 2, 4 (2009), 16:1–16:42.

[27] Hai Jin, Na Wang, Dongxiao Yu, Qiang-Sheng Hua, Xuanhua Shi, and Xia Xie. 2018. Core Maintenance in Dynamic Graphs: A Parallel Approach Based on Matching. *IEEE Trans. Parallel Distributed Syst.* 29, 11 (2018), 2416–2428.

[28] Tommaso Lanciano, Atsushi Miyauchi, Adriano Fazzone, and Francesco Bonchi. 2024. A Survey on the Densest Subgraph Problem and its Variants. *ACM Comput. Surv.* 56, 8 (2024), 208:1–208:40.

[29] Jure Leskovec, Lada A Adamic, and Bernardo A Huberman. 2007. The Dynamics of Viral Marketing. *TWEB* 1, 1 (2007), 5–es.

[30] Rong-Hua Li, Jeffrey Xu Yu, and Rui Mao. 2014. Efficient Core Maintenance in Large Dynamic Graphs. *IEEE Trans. Knowl. Data Eng.* 26, 10 (2014), 2453–2465.

[31] Yuqi Li, Guosheng Zang, Chunyao Song, Xiaojie Yuan, and Tingjian Ge. 2024. Leveraging Semantic Information for Enhanced Community Search in Heterogeneous Graphs. *Data Sci. Eng.* 9, 2 (2024), 220–237.

[32] Xuankun Liao, Qing Liu, Jiaxin Jiang, Byron Choi, Bingsheng He, and Jianliang Xu. 2025. Accelerating D-Core Maintenance over Dynamic Directed Graphs. In *ICDE*. 696–709.

[33] Boge Liu, Fan Zhang, Chen Zhang, Wenjie Zhang, and Xuemin Lin. 2019. CoreCube: Core Decomposition in Multilayer Graphs. In *WISE (Lecture Notes in Computer Science)*. 694–710.

[34] Dandan Liu, Run-An Wang, Zhaonian Zou, and Xin Huang. 2024. Fast Multilayer Core Decomposition and Indexing. In *ICDE*. 2695–2708.

[35] Dandan Liu and Zhaonian Zou. 2023. gCore: Exploring Cross-layer Cohesiveness in Multi-layer Graphs. *Proc. VLDB Endow.* 16, 11 (2023), 3201–3213.

[36] Qing Liu, Xuliang Zhu, Xin Huang, and Jianliang Xu. 2021. Local Algorithms for Distance-generalized Core Decomposition over Large Dynamic Graphs. *Proc. VLDB Endow.* 14, 9 (2021), 1531–1543.

[37] Xudong Liu and Zhaonian Zou. 2023. Common-Truss-Based Community Search on Multilayer Graphs. In *ADMA (3) (Lecture Notes in Computer Science)*. 277–291.

[38] Yu Liu, Qi Luo, Yanwei Zheng, Zhipeng Cai, and Dongxiao Yu. 2025. Efficient traversal for core maintenance in large-scale dynamic hypergraphs. *WWW* 28, 4 (2025), 45.

[39] Chengyang Luo, Qing Liu, Yunjun Gao, and Jianliang Xu. 2025. Synergetic Community Search over Large Multilayer Graphs. *Proc. VLDB Endow.* 18, 5 (2025), 1412–1424.

[40] Wensheng Luo, Qiaoyuan Yang, Yixiang Fang, and Xu Zhou. 2023. Efficient Core Maintenance in Large Bipartite Graphs. *Proc. ACM Manag. Data* 1, 3 (2023), 208:1–208:26.

[41] Mathoverflow Network. 2026. https://snap.stanford.edu/data/sx-mathoverflow.html.

[42] Multilayer Networks. 2026. https://manliodedomenico.com/data.php.

[43] Ahmet Erdem Sariyüce, Bugra Gedik, Gabriela Jacques-Silva, Kun-Lung Wu, and Ümit V. Çatalyürek. 2013. Streaming Algorithms for k-core Decomposition. *Proc. VLDB Endow.* 6, 6 (2013), 433–444.

[44] Mauro Sozio and Aristides Gionis. 2010. The Community-search Problem and How to Plan a Successful Cocktail Party. In *KDD*. 939–948.

[45] Longxu Sun, Xin Huang, Zheng Wu, and Jianliang Xu. 2024. Efficient Cross-layer Community Search in Large Multilayer Graphs. In *ICDE*. 2959–2971.

[46] Zitan Sun, Xin Huang, Qing Liu, and Jianliang Xu. 2023. Efficient Star-based Truss Maintenance on Dynamic Graphs. *Proc. ACM Manag. Data* 1, 2 (2023), 133:1–133:26.

[47] Superuser Network. 2026. https://snap.stanford.edu/data/sx-superuser.html.

[48] Yifu Tang, Chengfei Liu, Lu Chen, Rui Zhou, and Jianxin Li. 2025. Finding Time-Proximity Communities in Temporal Heterogeneous Information Networks. *Proc. VLDB Endow.* 18, 13 (2025), 5740–5752.

[49] Han Wang, Renjie Sun, Yongye Li, Chen Chen, Xiaoyang Wang, and Ying Zhang. 2025. Efficient Maximal Frequent Clique Enumeration in Multilayer Networks. In *DASFAA (3) (Lecture Notes in Computer Science)*. 169–185.

[50] Na Wang, Dongxiao Yu, Hai Jin, Chen Qian, Xia Xie, and Qiang-Sheng Hua. 2017. Parallel Algorithm for Core Maintenance in Dynamic Graphs. In *ICDCS*. 2366–2371.

[51] Run-An Wang, Dandan Liu, and Zhaonian Zou. 2024. FocusCore Decomposition of Multilayer Graphs. In *ICDE*. 2792–2804.

[52] Tongfeng Weng, Mo Sha, Xu Zhou, Jingjing Lu, Kenli Li, and Kian-Lee Tan. 2025. Efficient Temporal Edge-Core Maintenance in Streaming Graphs. *Proc. VLDB Endow.* 19, 4 (2025), 726–739.

[53] Qiaoyuan Yang, Wensheng Luo, Yixiang Fang, and Yuanyuan Zeng. 2026. Order-based Algorithms for Efficient Core Maintenance in Large Bipartite Graphs. *Proc. ACM Manag. Data* 4, 1 (2026), 1–26.

[54] Rongjin Yang, Zijun Chen, and Wenyuan Liu. 2026. Efficient Order-based Algorithms for Core Maintenance in Weighted Graph. *Inf. Sci.* 728 (2026), 122775.

[55] Yikai Zhang and Jeffrey Xu Yu. 2019. Unboundedness and Efficiency of Truss Maintenance in Evolving Graphs. In *SIGMOD Conference*. 1024–1041.

[56] Yikai Zhang, Jeffrey Xu Yu, Ying Zhang, and Lu Qin. 2017. A Fast Order-Based Approach for Core Maintenance. In *ICDE*. 337–348.

[57] Wei Zhou, Hong Huang, Qiang-Sheng Hua, Dongxiao Yu, Hai Jin, and Xiaoming Fu. 2021. Core Decomposition and Maintenance in Weighted Graph. *WWW* 24, 2 (2021), 541–561.

[58] Yuhan Zhou, Chuanyu Liu, Qing Liu, and Chengyang Luo. 2026. Core Maintenance on Dynamic Multilayer Graphs (technical report). https://github.com/ZJU-DAILY/MCM/blob/main/MCM-TechnicalReport.pdf.

[59] Rong Zhu, Zhaonian Zou, and Jianzhong Li. 2018. Diversified Coherent Core Search on Multi-Layer Graphs. In *ICDE*. 701–712.

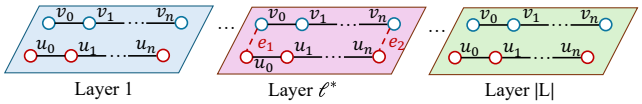


Figure 13: An example of boundedness analysis

A THE PROOF OF THEOREM 1

Theorem 1. *The multilayer core maintenance problem is unbounded under the model of LP algorithms.*

Proof. We prove that multilayer core maintenance is unbounded by constructing a counterexample that contradicts the boundedness assumption. Specifically, as shown in Figure 13, we construct a multilayer graph $\mathcal{G}$ containing two vertex-disjoint paths $\mathcal{P}_u = (u_0, u_1, \ldots, u_n)$ and $\mathcal{P}_v = (v_0, v_1, \ldots, v_n)$ on every layer, with two single-edge insertion updates $\Delta E_1 = \{e_1\}$ and $\Delta E_2 = \{e_2\}$ at opposite ends of the paths. When either update is applied individually, every vertex is still contained in a $\mathbf{k}$-core with $\mathbf{k} = [1]_{\ell \in L}$. Therefore, the MCV set of each vertex remains $\{\mathbf{1}\}$, where $\mathbf{1} = [1]_{\ell \in L}$. Besides, CHANGED contains only the endpoints of the updated edge and $|N_c(\text{CHANGED}, G^{\cup})| = O(1)$. By assumption, the cost of these two updates is $O(1)$. However, when the two updates are applied sequentially, they form a cycle on the updated layer ℓ^*, causing the MCV set of every vertex to change from $\{\mathbf{1}\}$ to $\{\mathbf{1} + \mathbf{e}_{\ell^*}\}$, where $\mathbf{e}_{\ell^*}$ is the unit vector whose ℓ^*-th dimension is one and all other dimensions are zero. Since applying ΔE_2 to $\mathcal{G}$ and to $\mathcal{G} \cup \Delta E_1$ produces different maintenance results, the algorithm must reach some vertex whose stored local state has been modified when applying ΔE_1 to $\mathcal{G}$. Therefore, the access traces for processing ΔE_1 and ΔE_2 individually on $\mathcal{G}$ must intersect; otherwise, the results of applying ΔE_2 to $\mathcal{G}$ and to $\mathcal{G} \cup \Delta E_1$ will be the same. The two traces together access $\Omega(n)$ local storage blocks for arbitrarily large n, implying that at least one individual update requires $\Omega(n)$ cost. Since n is not a constant value, it contradicts the boundedness assumption. Therefore, multilayer core maintenance is unbounded under the model of LP algorithms. □

B THE PROOF OF LEMMA 1

Lemma 1 (Edge Insertion Bound). *Let $\mathcal{G}' = \mathcal{G} \cup \{e\}$, where e is inserted on layer ℓ. For any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$, there exists a coreness vector $\mathbf{k}$ of v in $\mathcal{G}$ such that $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$, where $\mathbf{e}_\ell$ is the unit vector whose ℓ-th dimension is one and all other dimensions are zero. In particular, since every coreness vector is dominated by some MCV, there exists an original MCV $\mathbf{h} \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{h} + \mathbf{e}_\ell \succeq \mathbf{k}'$.*

Proof. Since $\mathbf{k}'$ is an MCV of v in $\mathcal{G}'$, there exists a vertex set C with $v \in C$ such that $\mathcal{G}'[C]$ is a multilayer $\mathbf{k}'$-core. After removing the inserted edge e, the degree of any vertex in C decreases by at most one on layer ℓ and remains unchanged on every layer $\ell' \neq \ell$. Therefore, every vertex in C still has degree at least $k'_\ell - 1$ on layer ℓ and at least $k'_{\ell'}$ on every other layer $\ell' \neq \ell$ in $\mathcal{G}[C]$. This implies that $\mathcal{G}[C]$ is also a multilayer $\mathbf{k}'^{(\ell,-1)}$-core. Although $\mathbf{k}'^{(\ell,-1)}$ is a coreness vector of v in $\mathcal{G}$, it may not be maximal. Thus, there exists an MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{k} \succeq \mathbf{k}'^{(\ell,-1)}$. It follows that $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$. □

C THE PROOF OF LEMMA 2

Lemma 2 (Same-level Insertion Transition). *If there is an insertion transition from $(x, \mathbf{k})$ to $(y, \mathbf{r})$ for an insertion on layer ℓ, then $k_\ell = r_\ell$.*

Proof. By Definition 4, $\mathbf{r} = \mathbf{k} \wedge \mathbf{k}'$. Therefore, $\mathbf{k} \succeq \mathbf{r}$. Since no original MCV of x dominates $\mathbf{r} + \mathbf{e}_\ell$, we have $\mathbf{k} \not\succeq \mathbf{r} + \mathbf{e}_\ell$. Hence, we can infer that $k_\ell = r_\ell$. □

D THE PROOF OF LEMMA 3

Lemma 3 (Edge Insertion Affected Scope). *Let $\mathcal{G}' = \mathcal{G} \cup \{e\}$. For any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$, there exists a vector $\overline{\mathbf{k}} \in \Phi_{\mathcal{G}}(v) \cup \{\mathbf{r} + \mathbf{e}_\ell \mid \mathbf{r} \in \mathcal{A}_\Phi(e, v)\}$ such that $\overline{\mathbf{k}} \succeq \mathbf{k}'$.*

Proof. Let $e = (u, w, \ell)$ and consider any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$. If $\mathbf{k}'$ is dominated by an original MCV in $\Phi_{\mathcal{G}}(v)$, then the lemma holds directly. We now consider the remaining case where $\mathbf{k}'$ is not dominated by any original MCV of v. By Lemma 1, there exists an original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ such that $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$. Since $\mathbf{k}$ itself cannot dominate $\mathbf{k}'$, the dominance of $\mathbf{k}'$ must rely on the promotion on layer ℓ. Thus, the newly obtained support for $\mathbf{k}'$ must be introduced either by the inserted edge directly or by a propagated affected vector from a neighbor.

If the new support is introduced directly by the inserted edge, then v is an endpoint of e. Suppose that $v = u$ and w is the other endpoint of e. Let C be an updated multilayer $\mathbf{k}'$-core containing v. If the inserted edge directly supports $\mathbf{k}'$ of v in $\mathcal{G}'[C]$, then the other endpoint w is also in $\mathcal{G}'[C]$. Thus, $\mathbf{k}'$ is a coreness vector of w in $\mathcal{G}'$, and is dominated by some updated MCV $\mathbf{h}' \in \Phi_{\mathcal{G}'}(w)$. By Lemma 1, there exists an original MCV $\mathbf{h} \in \Phi_{\mathcal{G}}(w)$ such that $\mathbf{h} + \mathbf{e}_\ell \succeq \mathbf{h}' \succeq \mathbf{k}'$. Let $\mathbf{r} = \mathbf{k} \wedge \mathbf{h}$, we next verify that $\mathbf{r} + \mathbf{e}_\ell$ dominates $\mathbf{k}'$. For any layer $i \neq \ell$, $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$ and $\mathbf{h} + \mathbf{e}_\ell \succeq \mathbf{k}'$ imply $k_i \geq k'_i$ and $h_i \geq k'_i$, so $r_i = \min\{k_i, h_i\} \geq k'_i$. For layer ℓ, they imply $k_\ell + 1 \geq k'_\ell$ and $h_\ell + 1 \geq k'_\ell$, so $r_\ell + 1 = \min\{k_\ell, h_\ell\} + 1 \geq k'_\ell$. Thus, $\mathbf{r} + \mathbf{e}_\ell \succeq \mathbf{k}'$. Since $\mathbf{k}'$ is not dominated by any original MCV of v, $\mathbf{r} + \mathbf{e}_\ell$ is also not dominated by any original MCV of v. Hence, the endpoint initialization inserts $\mathbf{r}$ into $\mathcal{A}_\Phi(e, v)$.

Otherwise, the new support is propagated from a neighbor $x \in N_{\mathcal{G}'}(v)$ whose affected subspace vector $\mathbf{h}$ has already been promoted. Let $\mathbf{r} = \mathbf{k} \wedge \mathbf{h}$. As in the direct case, since both $\mathbf{h} + \mathbf{e}_\ell$ and $\mathbf{k} + \mathbf{e}_\ell$ dominate $\mathbf{k}'$, $\mathbf{r} = \mathbf{h} \wedge \mathbf{k}$ gives $\mathbf{r} + \mathbf{e}_\ell \succeq \mathbf{k}'$. Moreover, no original MCV of x dominates $\mathbf{r} + \mathbf{e}_\ell$; otherwise, the support from x would already exist before the insertion and would not be a newly propagated support. No original MCV of v dominates $\mathbf{r} + \mathbf{e}_\ell$ either; otherwise, $\mathbf{k}'$ would be dominated by an original MCV of v. Therefore, the pair $(x, \mathbf{h})$ satisfies the insertion transition to $(v, \mathbf{r})$ and $\mathbf{r}$ is included into $\mathcal{A}_\Phi(e, v)$. Applying the same argument recursively to $(x, \mathbf{h})$ traces the newly available support backward along insertion transitions. The recursion terminates at an affected vector generated at an endpoint of the inserted edge, because the inserted edge is the only source of new support. Therefore, $\mathbf{r} \in \mathcal{A}_\Phi(e, v)$ and $\mathbf{r} + \mathbf{e}_\ell$ dominates $\mathbf{k}'$. □

E THE PROOF OF LEMMA 4

Lemma 4 (Edge Deletion Bound). *Let $\mathcal{G}' = \mathcal{G} \setminus \{e\}$, where e is deleted from layer ℓ. For any vertex $v \in V$ and any original MCV*

$\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, *there exists an updated MCV* $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$ *such that* $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$.

Proof. Since $\mathbf{k}$ is an MCV of v in $\mathcal{G}$, there exists a vertex set C with $v \in C$ such that $\mathcal{G}[C]$ is a multilayer $\mathbf{k}$-core. After deleting e, the degree of any vertex in C decreases by at most one on layer ℓ and remains unchanged on every other layer. Thus, $\mathcal{G}'[C]$ is a multilayer $\mathbf{k}^{(\ell,-1)}$-core. Hence, $\mathbf{k}^{(\ell,-1)}$ is a coreness vector of v in $\mathcal{G}'$ and is dominated by some MCV $\mathbf{k}' \in \Phi_{\mathcal{G}'}(v)$, i.e., $\mathbf{k}' \succeq \mathbf{k}^{(\ell,-1)}$. Therefore, $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$. □

F THE PROOF OF LEMMA 5

Lemma 5 (Same-Level Deletion Transition). *If there is a deletion transition from* $(x, \mathbf{k})$ *to* $(y, \mathbf{k}')$ *for a deletion on layer* ℓ*, then* $k_\ell = k'_\ell$.

Proof. By the definition of deletion transition, $\mathbf{k} \succeq \mathbf{k}'$, so $k_\ell \geq k'_\ell$. If $k_\ell > k'_\ell$, then $\mathbf{k}^{(\ell,-1)} \succeq \mathbf{k}'$. Since $\mathbf{k} \in \Phi_{\mathcal{G}}(x)$, it contradicts the condition that $\nexists \mathbf{h} \in \Phi_{\mathcal{G}}(x), \mathbf{h}^{(\ell,-1)} \succeq \mathbf{k}'$. Therefore, $k_\ell = k'_\ell$. □

G THE PROOF OF LEMMA 6

Lemma 6 (Edge Deletion Affected Scope). *Let* $\mathcal{G}' = \mathcal{G} \setminus \{e\}$. *For any vertex* $v \in V$*, if an original MCV* $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ *may become invalid after deleting* e*, then* $\mathbf{k} \in \mathcal{A}_\Phi(e, v)$.

Proof. Assume that an MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ may become invalid after deleting e. If the invalidation of $\mathbf{k}$ is caused directly by the deleted edge, then v is an endpoint and the other endpoint of the deleted edge has an MCV dominating $\mathbf{k}$. Thus, $\mathbf{k}$ is contained in the affected vector set of v.

Otherwise, the invalidation of $\mathbf{k}$ must be caused by a neighbor x whose original MCV dominates $\mathbf{k}$ in the original graph but can no longer be guaranteed to dominate $\mathbf{k}$ after the deletion. Let $\mathbf{k}_x \in \Phi_{\mathcal{G}}(x)$ be an affected vector of x with $\mathbf{k}_x \succeq \mathbf{k}$. If there exists an MCV $\mathbf{h} \in \Phi_{\mathcal{G}}(x)$ such that $\mathbf{h}^{(\ell,-1)} \succeq \mathbf{k}$, then by Lemma 4, x would have an updated MCV dominating $\mathbf{h}^{(\ell,-1)}$, and hence still dominating $\mathbf{k}$. This contradicts the fact that the dominance relation from x may be lost. Therefore, no such $\mathbf{h}$ exists. Since $\mathbf{k}_x \succeq \mathbf{k}$ and no original MCV of x still dominates $\mathbf{k}$ after decreasing one on layer ℓ, there is a deletion transition from $(x, \mathbf{k}_x)$ to $(v, \mathbf{k})$. Applying the same argument recursively to $(x, \mathbf{k}_x)$ yields a sequence of deletion transitions, and the recursion terminates at a directly affected vector generated at an endpoint of the deleted edge. Thus, there exists a deletion affected path of e that reaches $(v, \mathbf{k})$. Therefore, $\mathbf{k} \in \mathcal{A}_\Phi(e, v)$. □

H THE PROOF OF LEMMA 7

Lemma 7 (Validation Rule). *If every vector* $\overline{\mathbf{k}} \in \overline{\Phi}_{\mathcal{G}}(v)$ *is valid for any* $v \in V$*, then* $\overline{\mathbf{k}}$ *is a coreness vector of* v *in* $\mathcal{G}$.

Proof. For any $\overline{\mathbf{k}} \in \overline{\Phi}_{\mathcal{G}}(v)$, let $S_{\overline{\mathbf{k}}} = \{u \in V \mid \exists \overline{\mathbf{k}}' \in \overline{\Phi}_{\mathcal{G}}(u), \overline{\mathbf{k}}' \succeq \overline{\mathbf{k}}\}$. Notably, $v \in S_{\overline{\mathbf{k}}}$. For any $u \in S_{\overline{\mathbf{k}}}$, there exists a valid vector $\overline{\mathbf{k}}' \in \overline{\Phi}_{\mathcal{G}}(u)$ with $\overline{\mathbf{k}}' \succeq \overline{\mathbf{k}}$. By validity, for each layer ℓ, u has at least $\overline{k}'_\ell \geq \overline{k}_\ell$ neighbors whose upper-bound sets contain vectors dominating $\overline{\mathbf{k}}'$, and hence also dominating $\overline{\mathbf{k}}$. Therefore, these neighbors belong to $S_{\overline{\mathbf{k}}}$ and the subgraph induced by $S_{\overline{\mathbf{k}}}$ is a multilayer $\overline{\mathbf{k}}$-core containing v. □

I THE PROOF OF LEMMA 8

Lemma 8 (Safe Refinement). *Suppose that for every vertex* $x \in V$*, every MCV of* x *is dominated by at least one vector in* $\overline{\Phi}_{\mathcal{G}}(x)$*. Given an invalid vector* $\overline{\mathbf{k}} \in \overline{\Phi}_{\mathcal{G}}(v)$ *and a safe refinement set* $\mathcal{R}(\overline{\mathbf{k}})$*, let* $\overline{\Phi}'_{\mathcal{G}}(v) = \overline{\Phi}_{\mathcal{G}}(v) \setminus \{\overline{\mathbf{k}}\} \cup \mathcal{R}(\overline{\mathbf{k}})$*. Then every MCV of* v *is dominated by at least one vector in* $\overline{\Phi}'_{\mathcal{G}}(v)$.

Proof. Consider any MCV $\mathbf{k}^* \in \Phi_{\mathcal{G}}(v)$. By the definition of $\overline{\Phi}_{\mathcal{G}}(v)$, there exists a vector $\mathbf{h} \in \overline{\Phi}_{\mathcal{G}}(v)$ such that $\mathbf{h} \succeq \mathbf{k}^*$. If $\mathbf{h} \neq \overline{\mathbf{k}}$, then $\mathbf{h}$ remains in $\overline{\Phi}'_{\mathcal{G}}(v)$, so $\mathbf{k}^*$ is still dominated by at least one vector in $\overline{\Phi}'_{\mathcal{G}}(v)$; otherwise, $\mathbf{h} = \overline{\mathbf{k}}$, i.e., $\overline{\mathbf{k}} \succeq \mathbf{k}^*$. Since the invalid vector $\overline{\mathbf{k}}$ cannot be an MCV $\mathbf{k}^*$, $\overline{\mathbf{k}} \neq \mathbf{k}^*$. By the definition of safe refinement, there exists a vector $\overline{\mathbf{k}}' \in \mathcal{R}(\overline{\mathbf{k}})$ such that $\overline{\mathbf{k}}' \succeq \mathbf{k}^*$. Since $\overline{\mathbf{k}}' \in \overline{\Phi}'_{\mathcal{G}}(v)$, $\mathbf{k}^*$ remains dominated after the refinement. Therefore, $\mathbf{k}^*$ is still dominated by at least one vector in $\overline{\Phi}'_{\mathcal{G}}(v)$. □

J THE PROOF OF LEMMA 9

Lemma 9 (Dominance Inheritance). *Consider an upper-bound vector of* v *refined from* $\overline{\mathbf{k}}$ *to* $\overline{\mathbf{k}}^{new}$*. For any neighbor* u *of* v*, every* $\overline{\mathbf{k}}' \in Dom(\overline{\mathbf{k}}, v, u)$ *satisfies* $\overline{\mathbf{k}}' \succ \overline{\mathbf{k}}^{new}$.

Proof. By refinement rule, $\overline{\mathbf{k}} \succ \overline{\mathbf{k}}^{new}$. Since any vector $\overline{\mathbf{k}}' \in Dom(\overline{\mathbf{k}}, v, u)$ satisfies $\overline{\mathbf{k}}' \succeq \overline{\mathbf{k}}$, we have $\overline{\mathbf{k}}' \succ \overline{\mathbf{k}}^{new}$ by transitivity. Hence, every vector in $Dom(\overline{\mathbf{k}}, v, u)$ dominates $\overline{\mathbf{k}}^{new}$. □

K THE PROOF OF LEMMA 10

Lemma 10 (Repeated Vector Filtering). *The ordered splitting rule generates each vector at most once.*

Proof. Consider any vector generated from an initial invalid vector. It can be uniquely described by a decreasing vector $\mathbf{d} = (d_1, \ldots, d_{|L|})$, where d_i denotes how many times the value on layer i has been decreased. That is, once $\mathbf{d}$ is fixed, the resulting vector is also fixed. Under the ordered splitting rule, decreases must be applied in non-decreasing order of layer indices. Therefore, for a given decreasing vector $\mathbf{d}$, there is only one valid splitting path: first decrease layer 1 for d_1 times, then layer 2 for d_2 times, and so on until layer $|L|$. Any different order would apply a decrease to some layer j before a later decrease to a smaller layer $i < j$, which violates the ordered splitting rule. Hence, each decreasing vector can be generated by at most one valid splitting path. Since each generated vector is uniquely determined by its decreasing vector, the ordered splitting rule generates each vector at most once. □

L THE PROOF OF LEMMA 11

Lemma 11 (Validity Hierarchy). *Consider two upper-bound vectors* $\overline{\mathbf{k}}$ *and* $\overline{\mathbf{k}}'$ *of the same vertex, where* $\overline{\mathbf{k}} \succeq \overline{\mathbf{k}}'$*. If* $\overline{\mathbf{k}}$ *is valid,* $\overline{\mathbf{k}}'$ *is valid. If* $\overline{\mathbf{k}}'$ *is invalid,* $\overline{\mathbf{k}}$ *is invalid.*

Proof. Assume that $\overline{\mathbf{k}}$ is valid. By the validation rule, for every layer $\ell \in L$, there exist at least $\overline{k}_\ell$ neighbors on layer ℓ whose upper-bound vectors dominate $\overline{\mathbf{k}}$. Therefore, these neighbors also support $\overline{\mathbf{k}}'$ and $\overline{\mathbf{k}}'$ is valid. Suppose that $\overline{\mathbf{k}}'$ is invalid. If $\overline{\mathbf{k}}$ were valid, $\overline{\mathbf{k}}'$ would also be valid as proved above, which contradicts the assumption that $\overline{\mathbf{k}}'$ is invalid. Therefore, $\overline{\mathbf{k}}$ must be invalid. □

M THE PROOF OF LEMMA 12

Lemma 12 (Subtree Dominance). *Consider a vector $\overline{\mathbf{k}}$ in a splitting tree of the refinement phase and two child subtrees T_i and T_j generated by decreasing the value of $\overline{\mathbf{k}}$ on layers i and j with $i < j$, respectively. Then no vector in T_i dominates any vector in T_j. In contrast, a vector in T_j may dominate a vector in T_i.*

Proof. Every vector in T_i is generated by a splitting sequence that first decreases layer i. Thus, for any $\mathbf{x} \in T_i$, we have $x_i < \overline{k}_i$. By the ordered splitting rule, every vector in T_j can only decrease value on the dimension with indices at least j. Since $i < j$, the value on the i-th dimension cannot be decreased in T_j. Hence, for any $\mathbf{y} \in T_j$, we have $y_i = \overline{k}_i$. Therefore, for any $\mathbf{x} \in T_i$ and $\mathbf{y} \in T_j$, we have $x_i < y_i$, which implies $\mathbf{x} \not\succeq \mathbf{y}$. Next, we show that reverse dominance may occur. Let $a_i < \overline{k}_i$ and $a_j < \overline{k}_j$ be two decreased dimension values. Consider a vector $\mathbf{y} \in T_j$ with $y_j = a_j$ and $y_\ell = \overline{k}_\ell$ for every $\ell \neq j$. Let $\mathbf{x} \in T_i$ be a vector obtained by first decreasing the value on layer i to a_i and later decreasing the value on layer j to a_j, while keeping the remaining dimensions unchanged. Then $\mathbf{y}$ and $\mathbf{x}$ have the same values on all layers except layer i. Besides, $y_i = \overline{k}_i > a_i = x_i$. Therefore, $\mathbf{y} \succeq \mathbf{x}$. □

N THE PROOF OF LEMMA 13

Lemma 13 (Subspace Coverage). *If the splitting tree rooted at $\overline{\mathbf{r}}$ has been fully explored, then no vector in $\mathcal{D}(\overline{\mathbf{r}})$ can produce a new maximal valid vector.*

Proof. Any descendant of a vector $\overline{\mathbf{k}} \in \mathcal{D}(\overline{\mathbf{r}})$ is also dominated by $\overline{\mathbf{r}}$, and is therefore also included in $\mathcal{D}(\overline{\mathbf{r}})$. Since the splitting tree rooted at $\overline{\mathbf{r}}$ has been fully explored, all maximal valid vectors in this subspace have already been found. Thus, every valid vector generated from $\overline{\mathbf{k}}$ is either one of these maximal valid vectors or is dominated by one of them, and cannot become a new maximal valid vector. □

O THE PROOF OF LEMMA 14

Lemma 14 (Disjoint Edge Group Bound). *Let $B \subseteq \Delta E_\ell$ be a disjoint edge group updated on layer ℓ. If B is inserted, then for any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}\cup B}(v)$, there exists a coreness vector $\mathbf{k}$ of v in $\mathcal{G}$ such that $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$. If B is deleted, then for any vertex $v \in V$ and any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}\setminus B}(v)$ such that $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$.*

Proof. For insertion, consider any $\mathbf{k}' \in \Phi_{\mathcal{G}\cup B}(v)$ and a vertex set C containing v such that $(\mathcal{G}\cup B)[C]$ is a multilayer $\mathbf{k}'$-core. Since B is a disjoint edge group, each vertex in C is incident to at most one inserted edge in B. After removing B, the degree of every vertex decreases by at most one on layer ℓ and remains unchanged on all other layers. Thus, $\mathcal{G}[C]$ is a multilayer $\mathbf{k}'^{(\ell,-1)}$-core and $\mathbf{k}'^{(\ell,-1)}$ is a coreness vector of v in $\mathcal{G}$. Since $\mathbf{k}'^{(\ell,-1)}$ may not be maximal, there exists a coreness vector $\mathbf{k}$ of v in $\mathcal{G}$ such that $\mathbf{k} \succeq \mathbf{k}'^{(\ell,-1)}$. Hence, $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$.

For deletion, consider any $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ and a vertex set C containing v such that $\mathcal{G}[C]$ is a multilayer $\mathbf{k}$-core. Since B is a disjoint edge group, after deleting B, the degree of every vertex in C decreases by at most one on layer ℓ and remains unchanged on all other layers. Therefore, $(\mathcal{G} \setminus B)[C]$ is a multilayer $\mathbf{k}^{(\ell,-1)}$-core and $\mathbf{k}^{(\ell,-1)}$ is a coreness vector of v in $\mathcal{G} \setminus B$, which indicates that $\mathbf{k}^{(\ell,-1)}$ is dominated by some $\mathbf{k}' \in \Phi_{\mathcal{G}\setminus B}(v)$. Hence, $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$. □

P THE PROOF OF LEMMA 15

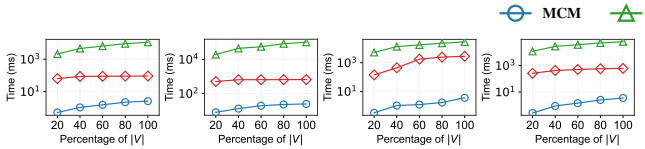
Lemma 15 (Joint Edge Group Bound). *Let $B \subseteq \Delta E_\ell$ be a joint edge group. If B is inserted, then for any vertex $v \in V$ and any updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}\cup B}(v)$, there exists a coreness vector $\mathbf{k}$ of v in $\mathcal{G}$ such that $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$. If B is deleted, then for any vertex $v \in V$ and any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$, there exists an updated MCV $\mathbf{k}' \in \Phi_{\mathcal{G}\setminus B}(v)$ such that $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$.*

Proof. We first consider insertion. For any common endpoint $w \in C_B$ and any incident edge $(u, w, \ell) \in B$, let $\mathbf{h} \in \Phi_{\mathcal{G}}(w)$, $\mathbf{q} \in \Phi_{\mathcal{G}}(u)$, and $\mathbf{r} = \mathbf{h} \wedge \mathbf{q}$. By Definition 8, we have $h_\ell \geq \mu_\ell(w) > \eta_\ell(u) \geq q_\ell$. Hence, $r_\ell = q_\ell$ and $\mathbf{h} \succeq \mathbf{r} + \mathbf{e}_\ell$. Since $\mathbf{r} + \mathbf{e}_\ell$ is dominated by an original MCV of w, w cannot become an affected vertex under the insertion transition. Therefore, the MCVs of w will not be affected by inserting edges in B. Now consider any $\mathbf{k}' \in \Phi_{\mathcal{G}\cup B}(v)$ and a vertex set C containing v such that $(\mathcal{G} \cup B)[C]$ is a $\mathbf{k}'$-core. For each common endpoint $w \in C \cap C_B$, since the MCVs of w remain unchanged after inserting B, there exists an original MCV $\mathbf{h}_w \in \Phi_{\mathcal{G}}(w)$ such that $\mathbf{h}_w \succeq \mathbf{k}'$. Let C_w be a vertex set containing w such that $\mathcal{G}[C_w]$ is a $\mathbf{h}_w$-core. Then $\mathcal{G}[C_w]$ is also a $\mathbf{k}'$-core. Let $\widehat{C} = C \cup \bigcup_{w\in C\cap C_B} C_w$. After removing B, every vertex in $C \setminus C_B$ loses at most one incident edge on layer ℓ, because it is incident to at most one edge in B by the definition of C_B, and its degree on all other layers remains unchanged. Since $\mathcal{G}[C_w]$ is a $\mathbf{k}'$-core, $\mathcal{G}[\widehat{C}]$ is a $\mathbf{k}'^{(\ell,-1)}$-core containing v. Therefore, $\mathbf{k}'^{(\ell,-1)}$ is a coreness vector of v in $\mathcal{G}$. Since $\mathbf{k}'^{(\ell,-1)}$ may not be maximal, there exists a coreness vector $\mathbf{k}$ of v in $\mathcal{G}$ such that $\mathbf{k} \succeq \mathbf{k}'^{(\ell,-1)}$. Hence, $\mathbf{k} + \mathbf{e}_\ell \succeq \mathbf{k}'$.

For deletion, a common endpoint w can be directly affected by deleting (u, w, ℓ) only if some MCV of u dominates an MCV of w, which is impossible because every MCV of w has a larger value on layer ℓ than every MCV of u by Definition 8. Moreover, by Lemma 5, deletion transitions only appear between vectors with the same value on layer ℓ. Hence, the common endpoints remain unaffected. Now consider any original MCV $\mathbf{k} \in \Phi_{\mathcal{G}}(v)$ and a vertex set C containing v such that $\mathcal{G}[C]$ is a $\mathbf{k}$-core. For each common endpoint $w \in C\cap C_B$, since the MCVs of w remain unchanged after deleting B, there exists an updated MCV $\mathbf{h}'_w \in \Phi_{\mathcal{G}\setminus B}(w)$ such that $\mathbf{h}'_w \succeq \mathbf{k}$. Let C'_w be a vertex set containing w such that $(\mathcal{G} \setminus B)[C'_w]$ is a $\mathbf{h}'_w$-core. Then $(\mathcal{G} \setminus B)[C'_w]$ is also a $\mathbf{k}$-core. Let $\widehat{C} = C \cup \bigcup_{w\in C\cap C_B} C'_w$. After deleting B, every vertex in $C \setminus C_B$ loses at most one incident edge on layer ℓ, because it is incident to at most one edge in B, and its degree on all other layers remains unchanged. Since $(\mathcal{G} \setminus B)[C'_w]$ is a $\mathbf{k}$-core, $(\mathcal{G} \setminus B)[\widehat{C}]$ is a $\mathbf{k}^{(\ell,-1)}$-core containing v. Thus, $\mathbf{k}^{(\ell,-1)}$ is a coreness vector of v in $\mathcal{G}\setminus B$ and is dominated by some $\mathbf{k}' \in \Phi_{\mathcal{G}\setminus B}(v)$. Hence, $\mathbf{k}' + \mathbf{e}_\ell \succeq \mathbf{k}$. □

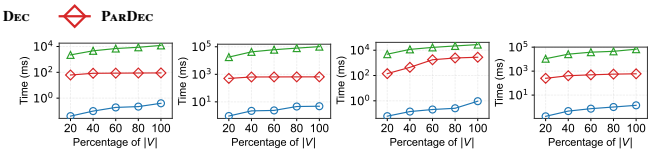
Q CORRECTNESS ANALYSIS OF ALGORITHM 3

We prove that Algorithm 3 terminates and returns the correct MCV set when validation follows Lemma 7 and each refinement is safe.



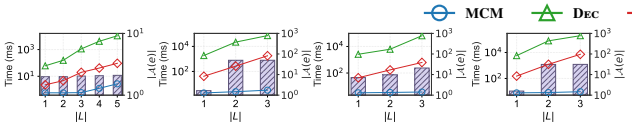
(a) AU (b) SU (c) YG (d) PO

Figure 14: Effect of graph size (insertion)



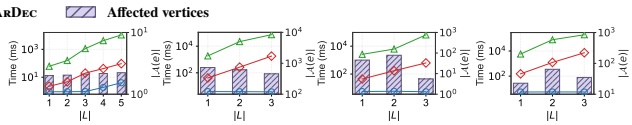
(a) AU (b) SU (c) YG (d) PO

Figure 15: Effect of graph size (deletion)



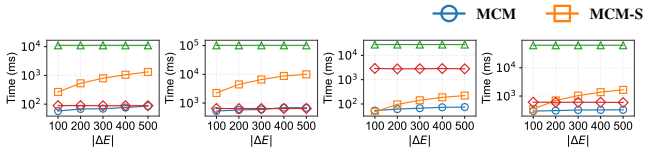
(a) AU (b) OB (c) PO (d) BT

Figure 16: Effect of the number of layers (insertion)



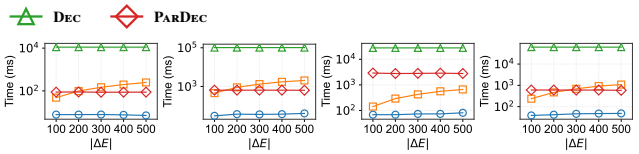
(a) AU (b) OB (c) PO (d) BT

Figure 17: Effect of the number of layers (deletion)



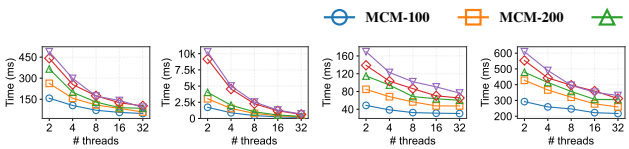
(a) AU (b) SU (c) YG (d) PO

Figure 18: Effect of batch updates (insertion)



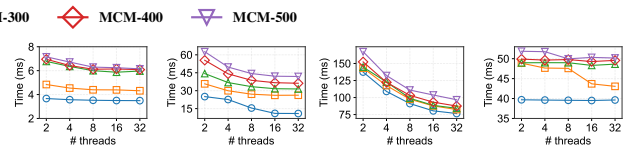
(a) AU (b) SU (c) YG (d) PO

Figure 19: Effect of batch updates (deletion)



(a) AU (b) SU (c) YG (d) PO

Figure 20: Effect of the number of threads (insertion)



(a) AU (b) SU (c) YG (d) PO

Figure 21: Effect of the number of threads (deletion)

Lemma 16 (Termination). *Algorithm 3 terminates after a finite number of updates.*

Proof. For each vertex v, let $\mathcal{D}_t(v) = \{\mathbf{k} \in \mathbb{N}^{|L|} \mid \exists \overline{\mathbf{k}} \in \overline{\Phi}_{\mathcal{G},t}(v),\ \overline{\mathbf{k}} \succeq \mathbf{k}\}$ be the subspace vector set of its upper-bound set $\overline{\Phi}_{\mathcal{G},t}(v)$ at time t. Initially, $\mathcal{D}_0(v)$ is finite. For each update, an invalid vector $\overline{\mathbf{k}}$ is replaced by a safe refinement set and all new vectors in the safe refinement set are strongly dominated by $\overline{\mathbf{k}}$. Therefore, no vector outside the subspace vector set is added, and $\overline{\mathbf{k}}$ is removed from this set. Thus, $\sum_{v \in \mathcal{A}(e)} |\mathcal{D}_t(v)|$ decreases after every update and is bounded below by zero. Finally, Algorithm 3 terminates after a finite number of updates. □

Lemma 17 (Maximality Preservation). *During Algorithm 3, for every vertex $v \in \mathcal{A}(e)$ and every MCV $\mathbf{k}^* \in \Phi_{\mathcal{G}}(v)$, there exists a vector $\overline{\mathbf{k}} \in \overline{\Phi}_{\mathcal{G}}(v)$ such that $\overline{\mathbf{k}} \succeq \mathbf{k}^*$.*

Proof. The claim holds initially by the definition of the upper-bound vector set. By Lemma 8, each refinement preserves this claim. Dominance filtering also preserves it because a filtered vector is dominated by a retained vector. The lemma follows by induction over all updates. □

Theorem 2 (Correctness). *Upon termination of Algorithm 3, $\overline{\Phi}_{\mathcal{G}}(v) = \Phi_{\mathcal{G}}(v)$ holds for every vertex $v \in V$.*

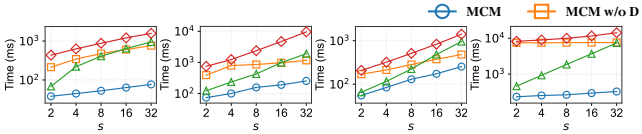
Proof. For every $v \notin \mathcal{A}(e)$, $\overline{\Phi}_{\mathcal{G}}(v) = \Phi_{\mathcal{G}}(v)$ holds by the affected scope identification procedure. For every $v \in \mathcal{A}(e)$, upon termination, every vector in every $\overline{\Phi}_{\mathcal{G}}(v)$ is valid. By Lemma 7, every retained vector is a valid coreness vector. For any $v \in \mathcal{A}(e)$ and $\mathbf{k}^* \in \Phi_{\mathcal{G}}(v)$, Lemma 17 gives a vector $\overline{\mathbf{k}} \in \overline{\Phi}_{\mathcal{G}}(v)$ with $\overline{\mathbf{k}} \succeq \mathbf{k}^*$. Since $\mathbf{k}^*$ should be maximal, we have $\overline{\mathbf{k}} = \mathbf{k}^*$. Finally, dominance filtering ensures that the vectors in $\overline{\Phi}_{\mathcal{G}}(v)$ remain mutually incomparable. Thus, $\overline{\Phi}_{\mathcal{G}}(v) = \Phi_{\mathcal{G}}(v)$ holds for every $v \in \mathcal{A}(e)$. □

R ADDITIONAL EXPERIMENTAL RESULTS

In this section, we provide additional experimental results on the remaining datasets.

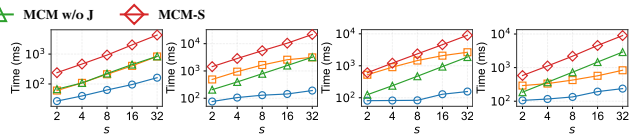
Effect of graph size. Figures 14 and 15 report the effect of graph size. MCM exhibits good scalability as the graph size increases under single-edge updates because its computation is restricted to the affected scope, which remains small relative to the entire graph and grows much more slowly than the graph size.

Effect of the number of layers. Figures 16 and 17 report the effect of the number of layers. Owing to its update locality, the running time of MCM remains relatively stable or increases only slightly as the number of layers grows. The number of affected vertices may vary non-monotonically with the number of layers. This is because, as more layers are included, updates can occur across a broader range of layers, resulting in different updated edges and affected



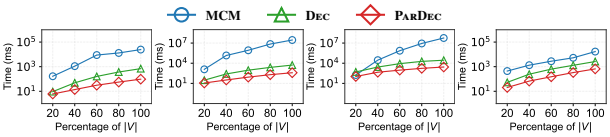
(a) AU (b) SU (c) YG (d) PO

Figure 22: Effect of hybrid edge grouping strategy (insertion)



(a) AU (b) SU (c) YG (d) PO

Figure 23: Effect of hybrid edge grouping strategy (deletion)



(a) AU (b) SU (c) YG (d) PO

Figure 24: Full-graph update time

scopes. Nevertheless, the number of affected vertices is typically much smaller than the graph size under the single-edge updates.

Effect of batch updates. Figures 18 and 19 report the performance under batch updates. Compared with MCM-S, the running time of MCM remains relatively stable or increases only slightly as the batch size grows. However, on some datasets, such as AU and SU, its running time may approach or even exceed that of ParDec for large update batches. This is because MCM is designed for real-time maintenance under frequent, small-scale updates, whereas ParDec performs decomposition only once after a large batch of updates. Therefore, when the affected scope and maintenance cost grow rapidly with the batch size, recomputing the MCVs using a decomposition-based method may be more efficient.

Effect of the number of threads. Figures 20 and 21 report the results with varying numbers of threads. The running time of MCM generally decreases as the number of threads increases. This reduction is particularly pronounced for larger update batches and settings with longer running times, where more computation can benefit from parallel execution.

Effect of the hybrid edge grouping strategy. Figures 22 and 23 evaluate the effect of the hybrid edge grouping strategy. MCM-S incurs a higher running time than all ablation variants, whereas the full MCM, integrating both disjoint and joint edge grouping, achieves the best overall performance. These results demonstrate the effectiveness of the two grouping strategies.

Performance of full-graph updates. Figure 24 reports the results under full-graph updates. The running time of MCM increases rapidly with the graph size because the affected scope expands substantially. Consequently, MCM must recompute MCVs from scratch through maintenance, which diminishes its advantage in update locality and introduces substantial additional maintenance overhead. This result further indicates that MCM is better suited to local updates than to full-graph computation.

S THE DETAILS OF BATCH UPDATES

Algorithm 4 shows the pseudocode of our multilayer core maintenance algorithm for batch-edge updates. The algorithm initializes the graph and MCV sets (line 1), and processes the updated edges layer by layer (line 2). For each layer, it repeatedly scans the remaining edges and greedily constructs a group B based on the hybrid grouping strategy (line 5). Then, the algorithm constructs

Algorithm 4: Multilayer Core Maintenance (Batch)

Input: A multilayer graph $\mathcal{G}$; input MCV sets $\Phi_{\mathcal{G}}(\cdot)$; updated edge set $\Delta E = \{\Delta E_\ell\}_{\ell \in L}$; operation $op \in \{\text{insert}, \text{delete}\}$
Output: Updated MCV sets $\Phi_{\mathcal{G}'}(\cdot)$

1 $\mathcal{G}_0 \leftarrow \mathcal{G}$; $\Phi_{\mathcal{G}_0}(\cdot) \leftarrow \Phi_{\mathcal{G}}(\cdot)$; $t \leftarrow 0$;
2 **foreach** $\Delta E_\ell \in \Delta E$ **do**
3 $R \leftarrow \Delta E_\ell$;
4 **while** $R \neq \emptyset$ **do**
5 $B \leftarrow$ apply hybrid grouping strategy on R; $R \leftarrow R \setminus B$;
6 $\mathcal{G}_{t+1} \leftarrow \mathcal{G}_t \cup B$ if op = insert; otherwise $\mathcal{G}_{t+1} \leftarrow \mathcal{G}_t \setminus B$;
7 $(\mathcal{A}(B), \overline{\Phi}_{\mathcal{G}_{t+1}}(\cdot)) \leftarrow$Identify affected scope of B;
8 Filter dominance relations in $\overline{\Phi}_{\mathcal{G}_{t+1}}(\cdot)$;
9 $\Phi_{\mathcal{G}_{t+1}}(\cdot) \leftarrow \text{LocalConvergence}(\mathcal{G}_{t+1}, \overline{\Phi}_{\mathcal{G}_{t+1}}(\cdot), \mathcal{A}(B))$;
  $t \leftarrow t + 1$;
10 **return** $\Phi_{\mathcal{G}'}(\cdot) \leftarrow \Phi_{\mathcal{G}_t}(\cdot)$;

the updated graph with B (line 6), and identifies the affected scope of updated edges (line 7). Next, it filters dominated upper-bound vectors (line 8) and invokes Algorithm 3 to refine the MCV sets (line 9). The process terminates after all updated edges have been processed and returns the updated MCV sets (line 10).

Complexity analysis. Let $m_\ell = |\Delta E_\ell|, \lambda = |L|, P = \max_{v \in V} |\Phi_{\mathcal{G}}(v)|$, g_ℓ be the number of groups constructed on layer ℓ, and g be the total number of groups in ΔE. Since Algorithm 4 scans the remaining edges on each layer in every grouping iteration, the total cost of constructing update groups is $O\left(\sum_{\ell \in L} g_\ell m_\ell\right)$. Hence, computing $\mu_\ell(\cdot)$ and $\eta_\ell(\cdot)$ for updated edges takes $O\left(\sum_{\ell \in L} g_\ell m_\ell P\right)$ time. For each update group B_i, let V_{B_i} denote the set of vertices visited during affected path traversal, and let $P_{B_i} = \max_{v \in V_{B_i}} \left|\bigcup_{e \in B_i} \mathcal{A}_\Phi(e, v)\right|$. According to the complexity analysis of Algorithm 2, identifying the affected scope for group B_i requires $O\left(\sum_{v \in V_{B_i}} d_v P_{B_i} P^2 \lambda\right)$ time. The complexity of local convergence has been analyzed in Algorithm 3. For simplicity, we denote its complexity for update group B_i as $T_{\text{conv}}(B_i)$. Therefore, the complexity of Algorithm 4 is $O\left(\sum_{\ell \in L} g_\ell m_\ell P + \sum_{i=1}^{g} \sum_{v \in V_{B_i}} d_v P_{B_i} P^2 \lambda + \sum_{i=1}^{g} T_{\text{conv}}(B_i)\right)$.